

AI and Machine Learning mathematics with algorithmic implementation

mohab metwally

2020

Contents

Introduction	11
0.1 notation	11
1 Logistic Regression as a neural network	13
1.1 definitions	13
1.2 cost function	13
1.3 Gradient Descent	14
1.4 Model training	15
1.5 Forward Propagation	16
1.5.1 Activation Functions	16
1.6 Backward Propagation	16
1.7 Update parameters	17
1.8 Summary	17
1.9 Logistic Regression in Python	18
1.10 References	18
2 Neural Networks	19
2.1 Lingua franca	19
2.2 Model training	20
2.3 Parameter initialization	21
2.3.1 Xavier initialization	22
2.4 Forward Propagation	22
2.5 Backward Propagation	22
2.6 Summary	23
2.7 Deep neural networks in Python	23
3 Neural Networks hyperparameters	27
3.0.1 training	27

3.0.2	bias-variance trade-off	28
3.0.3	recipes for high-bias, high-variance	28
3.0.4	Regularization (weight decay)	29
3.0.5	Inverted Dropout Regularization	30
3.0.6	Input Normalization	30
3.0.7	Vanishing/Exploding gradients	30
3.0.8	gradient checking	31
3.1	Optimization	32
3.1.1	stochastic, mini-batch, and batch gradient descent	32
3.2	Gradient descent with momentum	33
3.3	RMSprob	33
3.4	Adaptive Momentum estimation (Adam)	34
3.4.1	Hyperparameters	34
3.5	Learning rate decay	34
3.6	Tuning parameters	35
3.6.1	coarse to fine search	35
3.6.2	panda vs caviar training approaches	35
3.7	Batch Normalization	36
3.7.1	Covariate Shift	36
3.7.2	Batch normalization as a regularization technique	37
3.7.3	Batch normalization on test sets	37
3.8	Multi-class classification	37
3.8.1	Softmax activation	37
4	Classical Machine Learning	39
4.1	Support Vector Machine (SVM)	39
4.1.1	Large Margins in linear Decision Boundaries	42
4.1.2	Non-linear Decision Boundaries	42
4.1.3	Gaussian kernel	43
4.2	Unsupervised Learning (Introduction)	43
4.2.1	k-mean algorithm	43
4.2.2	Principle Component Analysis PCA	44
5	Structuring machine learning	47
5.1	Machine learning strategy	47
5.2	orthogonality	47
5.3	Set up your Goal	48
5.3.1	Evaluation metric	48

5.3.2	Precision vs accuracy	48
5.3.3	F1-score	49
5.3.4	Satisfying-Optimizing metric	49
5.4	train/dev/test sets	49
5.4.1	dev/test metric	50
5.5	Human-level performance	50
5.5.1	human(bayes) error	50
5.6	Error analysis	51
5.6.1	Mislabeled data	52
5.7	Mismatched training dev/test sets	52
5.7.1	Addressing data mismatch	53
5.7.2	Artificial data synthesis	53
5.8	Multiple tasks learning	54
5.8.1	Transfer learning	54
5.8.2	Multitask learning	54
5.9	End-to-End Machine Learning	54
6	Computer Vision	55
6.1	Object Detection	55
6.1.1	Edge Detection	56
6.1.2	Padding	57
6.1.3	striding	58
6.1.4	Convolution on RGB channels	58
6.1.5	Multiple filters Convolution	58
6.1.6	Example ConvNet	58
6.1.7	Pooling Convolutions	59
6.1.8	Max Pooling	59
6.1.9	Average Pooling	60
6.2	Examples	60
6.2.1	LeNet-5 Network	60
6.2.2	AlexNet Network	60
6.2.3	VGG-16 Network	61
6.2.4	ResNet (Residual Block)	62
6.3	Inception	62
6.3.1	1×1 Convolutions	63
6.3.2	Inception Block	63
6.3.3	Inception Branches	63
6.4	Object Detection	63

6.4.1	Localization and Detection	63
6.4.2	classification with localization	64
6.4.3	Landmark detection	64
6.4.4	Object Detection	65
6.4.5	Convolutional sliding windows	65
6.4.6	Example:	65
6.5	Bounding Box prediction	66
6.5.1	YOLO	66
6.5.2	Jaccard distance, or Intersection over Union IOU function	66
6.5.3	Non-Max Suppression	67
6.5.4	Anchor Boxes	67
6.5.5	Region Proposal R-CNN, Fast R-CNN	67
6.6	Face Recognition	68
6.6.1	Recognition vs. Verification	68
6.6.2	One-Shot learning	68
6.6.3	Similarity learning function	68
6.6.4	Siamese Network	69
6.6.5	Triplet Loss	69
6.6.6	Binary Classification	70
6.7	Neural Style Transfer	71
6.7.1	Generated image descent	71
6.7.2	Content cost	72
6.7.3	Style cost	72
6.7.4	Gram Matrix(style)	72
7	Recurrent Neural Networks RNN	73
7.1	Introduction	73
7.2	Notation	73
7.3	RNN	74
7.3.1	RNN model	74
7.4	uni-directional RNN Forward propagation	74
7.5	uni-directional RNN backward propagation through time	75
7.6	Variations of RNN models	75
7.6.1	many-to-one RNN	76
7.6.2	One-to-Many RNN	76
7.6.3	Many-to-Many of different input/output length	78
7.7	Language Model and sequence generation	78

7.8	Sampling novel sequences	79
7.9	Vanishing/Exploding gradients	79
7.9.1	Gated Recurrent Unit (GRU)	79
7.9.2	GRU simplified	80
7.9.3	Long Short Term Memory (LSTM)	81
7.9.4	Back-propagation	82
7.9.5	Bidirectional RNN	83
7.10	Deep RNNs	83
7.11	Word Representation	84
7.11.1	Word Embedding	84
7.11.2	Named Entity recognition	84
7.11.3	Learning word embedding	85
7.11.4	Word2Vec	86
7.11.5	Skip gram	86
7.11.6	hierarchical SOFTMAX	86
7.11.7	Negative Sampling	87
7.11.8	GloVe (global vector for word representation)	88
7.12	Example: Sentiment Classification	88
7.13	Sequence To Sequence Model	89
7.13.1	Beam search	90
7.13.2	normalization	90
7.13.3	Error Analysis	90
7.13.4	Bleu Score	91
7.14	Attention Model	91
7.14.1	Attention	92
8	Probabilistic Graphical Models (PGM)	93
8.1	Introduction	93
8.1.1	preliminaries	94
8.1.2	factors	94
8.2	Bayesian Network Fundamentals	95
8.2.1	constructing dependencies	95
8.3	Reasoning Patterns on BN	96
8.4	Flow of Probabilistic Influence	97
8.5	d-separation	97
8.5.1	I-map	98
8.6	Naive Bayes Model	99
8.7	Template Model	99

8.7.1	Distribution over Trajectories	100
8.7.2	Markov chain	100
8.7.3	Dynamic Bayesian Network DBN	101
8.7.4	Hidden Markov Model (HMM)	101
8.8	Structured CPD	102
8.8.1	Tree CPD	103
8.8.2	Multiplexer CPD	104
9	Natural language processing	105
9.1	pre-processing	105
9.2	Example: positive, negative classifier	105
9.3	Logistic regression classifier	106
9.4	Naive Bayes classifier	107
9.5	cosine similartis	109
9.5.1	Euclidean distance	110
9.6	Principle Component Analysis (PCA)	110
9.7	Machine Translation	111
9.7.1	Loss function L	112
9.7.2	gradient descent	113
9.7.3	fixed number of iterations	114
9.7.4	k-Nearest neighbors algorithm	114
9.7.5	Searching for the translation embedding	115
9.7.6	LSH and document search	115
9.7.7	Bag-of-words (BOW) document models	116
9.7.8	Choosing the number of planes	116
9.7.9	Getting the hash number for a vector	117
9.8	Probabilistic model of pronunciation and spelling	118
9.8.1	auto-correction	118
9.8.2	Bayesian inference model	119
9.8.3	Minimum edit distance	120
9.9	Grammar Weighted Automata	120
9.9.1	Markov chain	120
9.9.2	Hidden Markov Models HMMs	121
9.9.3	Viterbi Algorithm	122
9.9.4	Part of Speech tagging (POS)	123
9.10	N-grams	124
9.10.1	Smoothing	124
9.10.2	Back-off	124

<i>CONTENTS</i>	9
-----------------	---

9.10.3 Interpolation	125
--------------------------------	-----

Appendices	127
-------------------	------------

A Introduction to probabilities	129
--	------------

A.1 probabilities chain rule	129
--	-----

A.2 Naive Bayes	129
---------------------------	-----

B Covariance	131
---------------------	------------

C Single Value Decomposition	133
-------------------------------------	------------

D Exponentially weighted averages	135
--	------------

D.0.1 How to choose the value β ?	135
---	-----

E smoothing (add-k, and add-one Laplacian)	137
---	------------

F Kernels, and Convolution functions	139
---	------------

Introduction

0.1 notation

through the book i sometimes use subscript without braces, or superscript with braces, or even curly braces to indicate vector element access, or row access in the case of a matrix.

$(x, y) \in R^{n_x}, y \in 0, 1$ are the training data-set, where x is input, and y is corresponding output.

therefore $M = \{(x^1, y^1), (x^2, y^2), \dots, (x^m, y^m)\}$ is the training set, and M_{test} is test example.

X is the input of training set and $\in R^{n_x \times m}$, $X = [X^1 \ X^2 \ \dots \ X^m]$, $X^{(i)}$ is the i th column of length n , and can be written as x , or $x^{(i)}$.

Chapter 1

Logistic Regression as a neural network

1.1 definitions

we need an estimate function $\hat{y}$ for the input x , and weight parameters $w \in R^{n_x}, b \in R$.

logistic function is $\hat{y} = p(y = 1|x)$, and can be defined as follows: $\hat{y} = \sigma(w^T x + b)$, where the sigma function is defined by $\sigma(z) = \frac{1}{1+e^{-z}}$, and notice when $z \rightarrow \infty, \sigma = 1, z \rightarrow -\infty, \sigma = 0$.

1.2 cost function

starting with a estimation linear forward model $\hat{y}$, we calculate the difference between our estimate, and the real value y , and through optimization we try to minimize the difference, or loss/cost through gradient descent, then we update our model's parameters.

loss function is minimizing the difference between estimation $\hat{y}, y$, and can be defined as least square $L(\hat{y}, y) = \frac{1}{2}(\hat{y} - y)^2$, but least squares leads to non-convex loss function(with multiple local minimums).

there are different loss functions, but the most efficient is that which maximize the difference. we can define $P(y|x^{(i)}, \theta) = h(x^{(i)}, \theta)^{y^{(i)}}(1 -$

$h(x^{(i)}, \theta)^{1-y^{(i)}}$, to increase the sensitivity to the training set we take the likelihood function, as the loss, $L(\theta) = \prod_{i=1}^m P(y|x^{(i)}, \theta)$ see (**AppendixA**).

one final step in our model is that as m get larger L tend to go to zero, to solve this we define the average sum of log-likelihood, or loss function to be our Cost function.

we multiply by -1 since the sum of the log-likelihood function is negative.

the Cost function $J(\theta) = \frac{1}{m} \sum_{i=1}^m \log(h(x^{(i)}, \theta)^{y^{(i)}}(1 - h(x^{(i)}, \theta)^{1-y^{(i)}}))$

loss function is defined as $L(\hat{y}, y) = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$, $L \in [0 - 1]$.

cost function is defined as the average of loss function $J(w, b) = \frac{1}{m} \sum_{i=1}^m L(y^{(i)}, y)$

1.3 Gradient Descent

gradient descent is a way to tune the weighting parameters, the objective is the lean toward the fittest weights with respect to the least cost.

iterate through cost function **J** tuning with respect to weight parameters **w**, **b**.

iterate through: $w := w - \alpha \frac{\partial J}{\partial w}$, $b := b - \alpha \frac{\partial J}{\partial b}$, for tuning **w**, **b** for the least **J** possible, such that α is the learning rate of GD.

for simplicity $\partial J / \partial w$ replaced for ∂w , and similarly $\partial J / \partial b$ is replaced for ∂b .

forward propagation,

$$\partial w = \frac{\partial J}{\partial L} \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z} \frac{\partial z}{\partial w}$$

, similarly

$$\partial b = \frac{\partial J}{\partial L} \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z} \frac{\partial z}{\partial b}$$

.

$$\partial L / \partial \hat{y} = \frac{-y}{\hat{y}} + \frac{(1 - y)}{1 - \hat{y}}$$

,

$$\partial \hat{y} / \partial z = \frac{-e^{-z}}{1 + e^{-z}} = \hat{y}(1 - \hat{y}).$$

$$\partial L / \partial z = \hat{y} - y.$$

then we can deduce that the final iteration gradient descent step after calculating sigma, loss, and cost functions can be

$$w := w - \frac{\alpha}{m} \sum_{i=1}^m \frac{\partial L}{\partial b} = \frac{\alpha}{m} X^T (\hat{y} - y)$$

, and

$$b := b - \frac{\alpha}{m} \sum_{i=1}^m (\hat{y} - y)$$

.

1.4 Model training

to train a logistic regression model given data set of $\mathbf{X}, \mathbf{y}$ we divide it into 20% for testing, and 80% for training, such that the training set is used to train our model parameters, and testing set is a separate set to test our model's predictions.

we call $X = \{X_{training}, X_{testing}\}, y = \{y_{training}, y_{testing}\}$ the data set, and $\{X_{training}, y_{training}\}$ the training set, and $\{X_{testing}, y_{testing}\}$ the testing set.

using $X_{training}, y_{training}$ (for the rest of the chapter, and the book i will refer to them by X, y for simplicity) we start **Forward propagation** to estimate $\hat{y}$ calculate the difference between $y, \hat{y}$ through **Cost function** $J(y, \hat{y})$.

going backward to W, b we implement **Backward propagation** through $\frac{\partial J}{\partial w}$, and $\frac{\partial J}{\partial b}$.

finally we **update weight parameters** after sufficient iterations until we minimize our cost function completely.

1.5 Forward Propagation

we begin by initializing our weight, and bias parameters ω , b randomly.

1.5.1 Activation Functions

what is activation function?

sigmoid:

relu:

tanh:

Starting with input training set $\mathbf{X}$ in our model. we estimate

$$z = \omega^T X + b$$

. then

$$\hat{y} = \sigma(z)$$

.

calculate cost function

$$J(y, \hat{y})$$

.

1.6 Backward Propagation

after evaluating the cost function $J(y, \hat{y})$ we calculate it's derivative with respect to $\{\omega, b\}$.

$$\partial \omega = \frac{\partial L}{\partial \omega} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z} \frac{\partial z}{\partial \omega}$$

$$\frac{\partial L}{\partial \hat{y}} = -\left(\frac{y}{\hat{y}} - \frac{(1-y)}{(1-\hat{y})}\right) = \frac{\hat{y} - y}{\hat{y}(1-\hat{y})}$$

$$e^{-z} = \frac{1}{\hat{y}} - 1 = \frac{1 - \hat{y}}{\hat{y}}$$

$$\frac{\partial \hat{y}}{\partial z} = \frac{e^{-z}}{1 + e^{-z}} = (\hat{y})^2 e^{-z}$$

$$\partial \omega = X^T(\hat{y} - y)$$

$$\partial b = \hat{y} - y$$

1.7 Update parameters

we implement the following algorithm with a fixed number of iteration that is customized per application, and tuned by the Engineer, such that each application would require different tuning parameters from which is the iteration number.

we iterate the following: update the parameters ω , b , in the back propagation step using $\partial \omega$, ∂b .

$$\omega = \omega - \frac{\alpha}{m} X^T (y - \hat{y})$$

$$b = b - \frac{\alpha}{m} (y - \hat{y})$$

1.8 Summary

logistic regression fits parameters w, b so that $\hat{y} = \sigma(w^T x + b)$ estimates $P(y = 1|x)$. training minimizes the cross-entropy cost $J(w, b) = \frac{1}{m} \sum_i L(\hat{y}^{(i)}, y^{(i)})$ by gradient descent, whose vectorized gradients reduce to $\partial w = \frac{1}{m} X(\hat{y} - y)^T$ and $\partial b = \frac{1}{m} \sum_i (\hat{y}^{(i)} - y^{(i)})$, repeated until convergence.

1.9 Logistic Regression in Python

the whole model is a few lines of vectorized `numpy`: a sigmoid, one forward pass, one backward pass, and a gradient-descent loop.

```

1 import numpy as np
2
3 def sigmoid(z):
4     return 1.0 / (1.0 + np.exp(-z))
5
6 def train(X, y, alpha=0.01, iterations=2000):
7     # X: (n_x, m) features, y: (1, m) labels
8     n_x, m = X.shape
9     w = np.zeros((n_x, 1))
10    b = 0.0
11    for _ in range(iterations):
12        # forward propagation
13        yhat = sigmoid(np.dot(w.T, X) + b)          # (1, m)
14        # cost (cross-entropy)
15        cost = -np.mean(y * np.log(yhat) + (1 - y) * np.log(1
16            - yhat))
17        # backward propagation
18        dz = yhat - y                                # (1, m)
19        dw = np.dot(X, dz.T) / m                     # (n_x, 1)
20        db = np.sum(dz) / m
21        # update parameters
22        w -= alpha * dw
23        b -= alpha * db
24    return w, b
25
26 def predict(X, w, b):
27     return sigmoid(np.dot(w.T, X) + b) > 0.5

```

Listing 1.1: logistic regression with numpy

1.10 References

Andrew Ng, *Neural Networks and Deep Learning*, deeplearning.ai / Coursera.

this chapter follows the standard maximum-likelihood derivation of logistic regression; see also the appendix on the likelihood function.

Chapter 2

Neural Networks

2.1 Lingua franca

- **RELU Activation Function:** It turns out that using the Tanh function in hidden layers is far more better. (Because of the zero mean of the function). Tanh activation function range is $[-1,1]$ (Shifted version of sigmoid function). Sigmoid or Tanh function disadvantage is that if the input is too small or too high, the slope will be near zero which will cause us the gradient decent problem. RELU stands for rectified linear unit, it's a rectifier Activation function and can be defined as $f(x) = x^+ = \max(0, x)$ or $\begin{cases} 0 & x \leq 0 \\ x & x > 0 \end{cases}$ relu shows to be better replacement to sigmoid function σ for the reason that it help in avoiding the vanishing gradient problem.
- **Neuron:** is a linear regression algorithm denoted by z , or a , $z = W^T X + b$, such that W is the the weight vector of the network.
- **Shallow Layers:** also known as Hidden Layers, is a set of neurons, for example of the network of composed of input X , and output Y , with at least a single layer $L1$, and at most 2 layers, then the forward propagation will be as follows: we calculate the logistic function for the first layer (1), $z_i^1 = w^T X_i + b_i$, $\hat{Y}^{(1)} = \sigma(z_i^1)$, then we proceed to calculate the final logistic evaluation for the output layer with $\hat{Y}^{(1)}$ as an input instead of X , and so on we proceed replacing $\hat{Y}^{(i)}$ instead of X as new input.

- Layer: layer $L_{(i)}$ is $\hat{Y}^{(i)} = [\hat{y}_1^{(i)}, \hat{y}_2^{(i)}, \dots, \hat{y}_n^{(i)}]$ such that n is the length of the layer $L_{(i)}$. each $\hat{y}_j^{(i)}$ is weighted with unique weight vector with previous layer $L_{(i-1)}$.
- Neural Network(NN): is a set of interconnected layers, $\langle X, L_1, L_2, \dots, L_m, Y \rangle$
- Deepness: shallow layer as defined to consist of 1-2 hidden layers, but on the other hand Deep Network is consisting of more than 2 inner, or hidden layers.

we discussed in previous chapter that $\frac{\partial \hat{y}}{\partial z}$, is actually for the logistic activation function σ only, we need to calculate the same derivation for tanh, and RELU.

for $\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$ is $\frac{\partial \hat{y}}{\partial z} = 1 - \tanh(z)^2$

for relu activation function $\frac{\partial \hat{y}}{\partial z} = \begin{cases} 0 & z < 0 \\ 1 & z \geq 0 \end{cases}$

in NN there are plenty of parameters to worry about, for example the weights need to be initialized randomly, with small values, and b can be initialized as zero.

2.2 Model training

Training a deep neural network is analogous to training a logistic regression network, as discussed in previous chapter, a logistic regression model can be considered a neural network with zero **hidden layers**.

We follow the same algorithm, parameter initialization, but in this case we initialize the parameters for each layer, assume a network composed of 2 hidden layers $X \rightarrow L1 \rightarrow L2 \rightarrow Y$, layer $(L^{(i)}, L^{(i-1)})$ are interconnected with weight, bias parameters $\omega^{(1)}, b^{(1)}$, such that $\omega^{(i)}$ is a matrix of shape $(length(L^{(i)}), length(L^{(i-1)}))$, and b is a vector of shape $(length(L^{(i)}), 1)$, so each node in the layer $L^{(i)}$ is connected with each node in previous layer $L^{(i-1)}$.

W, b are ought to be randomly initialized, in the logistic regression discussed in previous chapter, W, b should be initialized with zero values, but in

the case of the neural network zero initialization leads to $\hat{y} = 0$, and all the nodes share the same weight which violates the purpose of the nodes in the first place which is to capture features from the data set as much as possible, so random initialization is necessary, and not to overshoot, initialization better in range $]0 - 1[$ weighted by small value around 0.01 (this will be discussed in details in NN hyperparameters chapter) to reduce the sensitivity, in order for the gradient descent to not take for ever.

forward and back propagation are executed in a similar way to logistic regression with minor differences.

calculation of forward propagation using inputs from the previous layer instead of input data set, activation function can be **sigmoid**, **relu**, or **tanh**, it has been tested that **relu** activation function in the first layers shows better results, and **sigmoid** activation in the last layer to fit perfectly with the output classification y-vector in the range $[0 - 1]$

2.3 Parameter initialization

Iterating through each layer, such that L^i layer of length l_i nodes, and previous layer L^{i-1} of l_{i-1} nodes, the weight parameter at layer L is inter-connected with all nodes in the previous layer, meaning that nodes at layer L^i ought to have weight matrix of shape (l_i, l_{i-1}) , and bias of shape $(l_i, 1)$.

Unlike the logistic regression, in Neural network initialization step is crucial step, in logistic regression there is only a single activation node extracting a single feature such as price of a house for example, but we intend to employ neural networks to capture as many features as possible inside each node, and therefore initialization ought to be with random values, otherwise we end up with similar copies of each node forward, and backward propagation.

and since we choose random W^i we can choose b^i to be zero.

2.3.1 Xavier initialization

randomizing weight parameters from a Gaussian, or normal distribution help to keep the initial values contained, or in other words, solves the vanishing, exploding gradient problem, instead to be demeaned around zero, this is proven to reduce variance (we will discuss this in details later on)

2.4 Forward Propagation

similar to logistic regression forward propagation, done for each layer, with little difference that instead of using sigmoid function σ we replace it with **relu** function, which shows better results, and activate the last layer with sigmoid function to distribute the output toward the extremes.

the forward propagation step is done on the input A^{i-1} from previous layer, with current layer L^i parameters ω^i, b^i .

we iterate through layers L^i such that $i \in [0, L]$

$$z^i = (\omega^i)^T A^{(i-1)} + b^i$$

$$A^i = \begin{cases} \text{relu}(z^i) \leftarrow \text{if } i < L \\ \sigma(z^i) \leftarrow \text{if } i = L \end{cases}$$

2.5 Backward Propagation

$$\frac{\partial L^i}{\partial \omega^i} = \frac{\partial L^i}{\partial A^{(i-1)}} \frac{\partial A^{(i-1)}}{\partial z^i} \frac{\partial z^i}{\omega^i}$$

$$\partial A^{(i-1)} = \partial z^i \left(\frac{\partial A^{(i-1)}}{\partial z^i} \right)^{-1} = \partial z^i \frac{\partial z^i}{\partial A^{(i-1)}}$$

since the last activation function is sigmoid, then

$$\partial z^L = y^L - \hat{y}^L$$

the back propagation algorithm start with the following initialization step:.

$$\partial A^{(L)} = \frac{(\hat{y} - y)}{\hat{y}(1 - \hat{y})}$$

$$\partial A^{(L-1)} = (\omega^{(L)})^T (y^{(L)} - \hat{y}^{(L)})$$

$$\frac{\partial z^i}{\partial A^{(i-1)}} = \omega^T$$

$$\partial A^{(i-1)} = \omega^T \partial z^i$$

2.6 Summary

a deep network stacks L layers, each applying an affine map followed by a non-linearity: $z^{[l]} = W^{[l]}A^{[l-1]} + b^{[l]}$, $A^{[l]} = g^{[l]}(z^{[l]})$, with **relu** in the hidden layers and **sigmoid** (or softmax) at the output.

training repeats three steps: a **forward pass** caching every $z^{[l]}, A^{[l]}$; a **backward pass** propagating $dz^{[l]} = dA^{[l]} * g'^{[l]}(z^{[l]})$, $dW^{[l]} = \frac{1}{m} dz^{[l]} A^{[l-1]T}$, $db^{[l]} = \frac{1}{m} \sum dz^{[l]}$, $dA^{[l-1]} = W^{[l]T} dz^{[l]}$; and a **parameter update** $W^{[l]} := W^{[l]} - \alpha dW^{[l]}$.

unlike logistic regression the weights must be initialized randomly (small values) so the hidden units learn distinct features.

2.7 Deep neural networks in Python

the same three steps generalize to L layers by iterating over a list of weight matrices; the forward pass caches activations that the backward pass consumes.

```

1 import numpy as np
2
3 def relu(z):          return np.maximum(0, z)
4 def relu_grad(z):     return (z > 0).astype(float)
5 def sigmoid(z):       return 1.0 / (1.0 + np.exp(-z))
6
7 def init_params(dims):
8     # dims = [n_x, n_1, ..., n_L]
9     params = {}
10    for l in range(1, len(dims)):
11        params["W" + str(l)] = np.random.randn(dims[l], dims[
12            l-1]) * 0.01
13        params["b" + str(l)] = np.zeros((dims[l], 1))
14    return params
15
16 def forward(X, params, L):
17     cache = {"A0": X}
18     A = X
19     for l in range(1, L + 1):
20         Z = np.dot(params["W" + str(l)], A) + params["b" +
21             str(l)]
22         A = sigmoid(Z) if l == L else relu(Z)    # sigmoid on
23             the output layer
24         cache["Z" + str(l)], cache["A" + str(l)] = Z, A
25     return A, cache
26
27 def backward(y, params, cache, L, m):
28     grads = {}
29     AL = cache["A" + str(L)]
30     dZ = AL - y                                # sigmoid +
31         cross-entropy
32     for l in range(L, 0, -1):
33         A_prev = cache["A" + str(l-1)]
34         grads["dW" + str(l)] = np.dot(dZ, A_prev.T) / m
35         grads["db" + str(l)] = np.sum(dZ, axis=1, keepdims=
36             True) / m
37         if l > 1:
38             dA_prev = np.dot(params["W" + str(l)].T, dZ)
39             dZ = dA_prev * relu_grad(cache["Z" + str(l-1)])
40     return grads
41
42 def train(X, y, dims, alpha=0.01, iterations=3000):
43     L = len(dims) - 1
44     m = X.shape[1]
45     params = init_params(dims)

```



```
41     for _ in range(iterations):
42         AL, cache = forward(X, params, L)
43         grads = backward(y, params, cache, L, m)
44         for l in range(1, L + 1):
45             params["W" + str(l)] -= alpha * grads["dW" + str(
46                 l)]
47             params["b" + str(l)] -= alpha * grads["db" + str(
48                 l)]
49     return params
```

Listing 2.1: an L-layer feed-forward network with numpy

Chapter 3

Neural Networks hyperparameters

machine learning have a wide applications in ranging from Computer Vision, Natural Language Processing, Speech recognition, and structured data, in data-science, and the the parameterization varies greatly from one domain to another, it's domain specific.

3.0.1 training

given a data set, wide range of models, the data data is divided into three categories, the **training**, **development**, and the **testing**.

we train different models to evaluate the most fit model on the development(also called evaluation set), after choosing our model we test on the testing set.

the partitioning ratio of the data-set varies with the size of the data set, for example in a small data set of size 1000, it's convenient to divide the data into 70%, 30% for training, and test respectively, or 60%, 20%, 20% for training, evaluating, and testing respectively, but in the era of big data, of millions of entries, data set is conveniently divided into (98%, 1%, 1%), (99.5%, 0.4%, 0.1%) for training, evaluation, testing sets respectively, the ratio varies from one domain to another.

for faster, and accurate training/testing the data-set ought to be for the same source of the same general range of features, on the other hand in

the mismatched data-set, in the case of training set from specific source, and development/test set from different set, the development/testing process can be inefficient, so the division of the partitioned sets need to be taken randomly from the same source.

3.0.2 bias-variance trade-off

In an n-dimensional training model, the bias-variance dimension indicates how far our model captures the data, and can be classified into under-fitting high-bias model where it hardly fit the data, on the other extreme, high-variance, over-fitting model, and in between just-right model.

high-variance: in a classification problem where the training set error is 1%, while the dev set error is 11%, this gape indicate that the model is well trained in a very narrow data-set with limited range of features quite different from that of the development set, in this case the model is over-fitting, or hardly scaling to other data-sets, and therefore fails in application.

high-bias: similarly in the same experiment if the training set error is around 15% where the dev-set error is 16%, indicating that the model generalize much better than the previous case of high-variance over-fitting model, but with high error would indicate that the model is highly-biased, and well generalized.

high-bias, high-variance: the worst model is that which do not generalize well, and doesn't fit our data, with high training error say 15%, and 30% development error.

low-bias, low-variance: the perfect model is that of low bias, low-variance, that which fits the training set, and generalizes well.

3.0.3 recipes for high-bias, high-variance

high-bias solution: getting a high bias means that our model doesn't capture sufficient features, and this can be due to our network is short, or narrow, or the gradient descend need to be run for longer time, or through using optimization, or change the neural network architecture.

high-variance solution: high variance indicate the lack of generalization, and is reduced through training on wider data-set, regularization, and changing the NN-architecture.

3.0.4 Regularization (weight decay)

in the case of high-variance, we need our model be more generalized over the data-set, one way to do so is through regularization, or changing the sensitivity of the weight parameters, this sensitivity is measured in the back-propagation step through $\partial J / \partial w$, we can vary the sensitivity by regularization parameter λ added to the cost function as follows:

$$J(w^1, w^2 \dots, w^L, b^L, b^L \dots, b^L) = \frac{\alpha}{m} \sum_{i=1}^m L(w^1, w^2 \dots, w^L, b^1, b^2 \dots, b^L) + \frac{\lambda}{2m} \sum_{l=1}^L \|w^l\|^2 + \frac{\lambda}{2m} b^L$$

but usually the last term is quite ineffective in regularization, and ignored, so it's cuts down to the following:

$$J(w^1, w^2 \dots, w^L, b^L, b^L \dots, b^L) = \frac{\alpha}{m} \sum_{i=1}^m L(w^1, w^2 \dots, w^L, b^1, b^2 \dots, b^L) + \frac{\lambda}{2m} \sum_{l=1}^L \|w^l\|^2$$

where the last norm is frobenius norm:

$$\sum_{i=1}^{n^{(l)}} \sum_{j=1}^{n^{(l-1)}} (w_{i,j}^{(l)})^2$$

backward propagation step:

$$\frac{\partial J}{\partial w^i} = \frac{1}{m} \sum_{i=1}^m \frac{\partial L}{\partial w^L} + \frac{\lambda}{m} \sum_{l=1}^L \|w^l\|$$

looking at our mechanism now, λ is a device for tuning the weight parameter, or rather as a valve to control it, the larger the λ is the lower the w as we minimize our cost function.

3.0.5 Inverted Dropout Regularization

instead of reducing the weight parameters, we can drop it out completely by assigning those values to zero.

implementation algorithm: by matching a threshold probability $= P_{keep}$ against activation matrix $A^{(i)}$, and to keep balanced activation values, it's re-evaluated as $A^{(i)}/(1 - P_{keep})$ and this last step distinguish the inverted dropout, from normal dropout, and the inverted is more efficient.

3.0.6 Input Normalization

since there are no constraint on the input features, each feature can be of different range, and the cost function can end up elongated at one dimension, and narrow on the other, being steep on one edge of the function, and smooth on the other, it can be visualized by taking a contour of the $\mathbf{J}$ function, it will look like very long ellipse.

with un-normalized input the gradient descend tends to take long time, and longer way down to the local minima, but if the input is normalized (demeaned, and divided by the standard deviation of the input) we end up with circular counter, and smooth, and faster gradient descent.

3.0.7 Vanishing/Exploding gradients

let's look deeper inside a deeper network, through L layers, such that L is large, $\hat{y} = (W^L W^{(L-1)} W^{(L-2)} \dots W^1) X$ if we assume $b = 0$ for simplicity, and that W^L is the transpose of w^L , in case entries of similar weight matrices in values, if the values of each W^i matrix below the 1, then the overall $(W^L W^{(L-1)} W^{(L-2)} \dots W^1)$ term will be very small, and gradient descent will take forever, on the other hand, if those entries greater than 1, then the overall term will be huge, and the gradient descend will overshoot(exploding).

a well chosen initialization parameters can Speed up the convergence of gradient descent, and Increase the odds of gradient descent converging to a lower training (and generalization) error.

to solve vanishing/exploding gradients problem we take two steps in weight parameters initialization, first the values are choosing at random through normal distribution, secondly, setting up the variance of w^i to $\frac{1}{k}$ such that k is the number of the neurons in layer $i - 1$. and it's by multiplying of w^i by $\sqrt{\frac{1}{k}}$, this ratio shows to fit better with relu activation if replaced with $\sqrt{\frac{2}{k}}$

the first variant $W^{[l]} \sim \mathcal{N}(0, 1) \cdot \sqrt{\frac{1}{n^{[l-1]}}}$ is **Xavier (Glorot) initialization**, derived so that the variance of the activations is preserved from one layer to the next for a tanh non-linearity; the relu variant $\sqrt{\frac{2}{n^{[l-1]}}}$ is **He initialization**, the extra factor of 2 compensating for relu zeroing out half of its inputs. keeping the per-layer variance near 1 is exactly what stops the product $W^{[L]} \dots W^{[1]}$ from decaying to zero or blowing up.

back to inverted dropout algorithm, due to the vanishing/exploding gradients problem we fix a threshold P_{keep} and compare each entry in W_i (taken from uniform distribution [0-1], unlike previously we choose W at random from normal/Gaussian distribution), so.

back propagation step we run the same masking algorithm over ∂A^l and divide it by the same factor.

if $A^l = \frac{mask(A^l)}{P_{keep}} = \frac{A^l * A_{mask}}{P_{keep}}$, then $\partial A^l = \frac{\partial A^l * A_{mask}}{P_{keep}}$ note that $*$ means element-wise multiplication.

3.0.8 gradient checking

we verify the GD isn't converging.

it's a method to verify that the gradient descent is within diminishing ϵ of error difference with the manual derivative of the derived function $f(\theta)$, and θ is a vector of unwrapped weight, and bias parameters.

if following the Euclidean distance L_2 norm difference $< (\epsilon = 10e - 7)$ then

the result is negative.

$$\frac{\left\| \frac{\partial J(\theta^i)}{\partial \theta^i} - \frac{\nabla J(\theta^i)}{\nabla \theta^i} \right\|}{\left\| \frac{\partial J(\theta^i)}{\partial \theta^i} \right\| + \left\| \frac{\nabla J(\theta^i)}{\nabla \theta^i} \right\|} < \epsilon$$

3.1 Optimization

machine learning is highly iterative process, to choose low-bias, low-variance model from the set of trained/tested models, the iteration process for each model through the same m data-set entries, of L layers, and this process is time consuming, optimized algorithms are essential.

3.1.1 stochastic, mini-batch, and batch gradient descent

let's consider the input data-set X of image-classification model Neural Network, using big-data of 5 million images, each with resolution of $800 * 400$, and three color channels, then our input Matrix X would be of size $5M * 800 * 400 * 3$ bytes need to be loaded in the training machine memory, that is around 4.37 Tera bytes!

to be able to train our model we divide our Input set into mini-batches, if we choose the batch size to be 1 meaning we run the gradient descent on a single input entry, without taking benefit of vectorization, and machine GPU, then it's called **stochastic gradient descent** it's a **mini-batch** of size 1, even using input from the same distribution the cost function hardly reduces to the local minima, and tend to take random-walks for each input, it's hard to predict it's behavior, but for the overall training-set it converges to the neighbourhood area of the local minima. but on the bright side it can run on a moderate computer, but takes longer time compared to batch gradient descent that make full use of vectorization, and GPU power.

if we take the benefits of both worlds and choose mini-batch size fit enough to run in our training machine GPU/CPU memory, we can run our model in the optimal time possible. so instead of running (X,Y) of

size m inside our model, we divide our training data-set (X, Y) into $(X^{\{t\}}, Y^{\{t\}})|_t = \frac{m}{\text{number-of-batches}}$ mini-batches.

3.2 Gradient descent with momentum

In the case of min-batch gradient descent, or in slow gradient descent the descent tends to overshoot, and deviate from the shortest path, to solve overshooting we employ weighted averages of the gradients (see appendix (D) on exponentially weighted averages) as follows:

$$Vdw^i = \beta_1 Vdw^{i-1} + (1 - \beta_1)dw^i$$

then we use Vdw^i in update parameters step, $w^i = w^i - \alpha Vdw^i$.

3.3 RMSprob

In a highly oscillating converging gradient descent around the axis of convergence, we can employ 'root mean squared prob' algorithm also using weighted averages, and here is the formula

$$Sdw^i = \beta_2 Sdw^{i-1} + (1 - \beta_2)(dw^i)^2$$

, the only difference here is that the last term's parameter is squared, and it's useful in the case of oscillating gradient along side an axis perpendicular to the converging axis, the squared weight will boost relative large value, or shrink down relatively small value, then in the update step we divide by the square root of the calculated maximized/shrunked weight value, if it was initialize relatively small we can compensate by increasing the learning rate α to leap larger steps, if the initial dw value was relatively large then we shrink it for more stable gradient descent.

the last step is:

$$w^i = w^i - \alpha \frac{dw^i}{\sqrt{Sdw^i + \epsilon}}$$

3.4 Adaptive Momentum estimation (Adam)

It's not always the case that one optimization algorithm on first application will scale will on a the second, so combination of different optimization algorithms shows to work best.

adam algorithm combines both RMSprob, and momentum gradient descent:

$$w^i = w^i - \alpha \frac{Vdw_{corrected}^i}{\sqrt{Sdw_{corrected}^i + \epsilon}}$$

$$b^i = b^i - \alpha \frac{Vdb_{corrected}^i}{\sqrt{Sdb_{corrected}^i + \epsilon}}$$

(look up appendix D for further explanations on bias correction)

3.4.1 Hyperparameters

the overall number of parameters is increasing:

- α : need to be tuned.
- β_1 : set to 0.9 (can be tuned).
- β_2 : set to 0.99 (can be tuned).
- ϵ : set to 10^{-8} .

3.5 Learning rate decay

at the start of gradient descent, it start making large leaps away from the maximum minima, but as it converges, gradient can converge faster if the α is tuned to adjust itself to get smaller over time.

in gradient descent iterations are called epochs, of index $epoch_{num}$ an epoch with rate $decay_{rate}$, we initialize learning rate to relatively large value α_0 then we set our decaying learning rate to:

$$\frac{1}{1 + epoch_{num} * decay_{rate}} \alpha_0$$

there are different variation of learning rate decay of the form $scale * \alpha_0$ for example:

$$0.95^{epoch_{rate}} \alpha_0$$

3.6 Tuning parameters

so far our highly optimized neural network is composed of many parameters left to the designer to tune (ordered by priority):

1. learning rate α .
2. number of hidden unites.
3. momentum averaging constant β .
4. mini-batch size.
5. number of layers $\mathbf{L}$.
6. learning rate decay initial value α_0 .
7. ADAM's $\beta_1, \beta_2, \epsilon$.

so we have 7 parameters so far, the permutation of those 7 variables even in a narrow range of discrete values, can take for ever, and an old-practice was to pick random permutation on a grid, meaning that *hyper-parameter_i*, against *hyper-parameter_j*, such that each cell map two different parameters, but since we are solving a stochastic search problem, it's more efficient to set the values at random in both dimension so for example for each *hyper-parameter_i* is projected on the *hyper-parameter_j* at each point, and tested, this opens more possibilities.

3.6.1 coarse to fine search

3.6.2 panda vs caviar training approaches

depending on the computational capacity available you choose between panda, and caviar approaches, training a single model at a time, and tuning it's parameters every set of epochs in case you have limited computational resources is called panda approach, on the other hand, if you have sufficient power to run different models with different parameters simultaneously, this last approach is called caviar (those terms were coined by Andrew Ng in a coursera specialization).

3.7 Batch Normalization

in previous chapter we discussed input normalization, for the same reasons we can implement the same method for each layer's input before the activation process, then we re-scale by λ, β :

$$z_{norm}^{[l]} = \frac{z^{[l]} - \mu(z^{[l]})}{\sqrt{\sigma^2 + \epsilon}} | l \in [1 - L]$$

$$\widetilde{z}^{[l]} = \lambda z_{norm}^{[l]} + \beta$$

but make sure here that $(\beta, \lambda) \neq (\sqrt{\sigma^2 + \epsilon}, \beta)$, because in such case we end up with $\widetilde{z}^{[l]} = z_{norm}^{[l]}$. here λ, β are learned parameters, and updated during the training.

update λ, β in the update step:

$$\beta^{[l]} = \beta^{[l]} - \alpha d\beta^{[l]}$$

$$\lambda^{[l]} = \lambda^{[l]} - \alpha d\lambda^{[l]}$$

in case of implementation of batch normalization we end up with the following:

$$z_{norm}^{[l]} = \frac{z^{[l]} - \mu(z^{[l]})}{\sqrt{\sigma^2 + \epsilon}} | l \in [1 - L]$$

, and

$$\widetilde{z}^{[l]} = \lambda z_{norm}^{[l]} + \beta$$

if we look closely we found that β also serves the purpose of bias, or shifting term $\mathbf{b}$ in forward algorithm: $z^{[l]} = w^T X^{\{i\}} + b$, therefore we can set $\mathbf{b}$ to zero, or remove this term.

3.7.1 Covariate Shift

if our data-set was from specific distribution, if we cut through the neural network at layer l , then all inputs $a^{[l-1]}$ feeding layer l are depended on the tuned parameters, gradients, and basically our input data, hence if $a^{[l]}$ are projected in specific neighbourhood for mini-batch $X^{\{i\}}$, at different mini-batch $X^{\{j\}}$ the projection can shift greatly in different

area. Batch normalization help to muzzle this overshoots, by mapping the output of previous layer $l - 1$ into normalization dimension, hence keeping the output more controlled for different mini-batches.

3.7.2 Batch normalization as a regularization technique

analogous to regularization, each mini-batch is scaled by mean, and variance, with the effect of randomizing the network layers output, similar to dropout technique, therefor it can be considered as regularization technique.

3.7.3 Batch normalization on test sets

during the test time, you can run the model on a a single example, so there are two approaches for calculation of μ, σ^2 those values can be taken from the final mini-batch, or through exponential/running weighted averages run over each mini-batch parameters.

3.8 Multi-class classification

so far we discussed a **sigmoid** activation over $\hat{y}$, what if our model's $\mathbf{y}$ is multi-class, and $\mathbf{y}$ isn't a single number but a vector of length n ?

if we run for example sigmoid, or relu on a vector rather than a real number the sum of the vector entries will be arbitrary number,

3.8.1 Softmax activation

softmax is a probabilistic activation function, run over $z^{[L]}$ is defined as:

$$a^{[L]} = \frac{e^{z^{[L]}}}{\sum_{i=1}^{n^{[L]}} t_i},$$

where $t = e^{z^{[L]}}$.

the resultant $\sum_{i=1}^{n^{[L]}} a_i^{[L]} = 1$, where $n^{[L]}$ is the number of neurons, or nodes in the final layer, which is the same as the number of classes at output $\mathbf{y}$.

Chapter 4

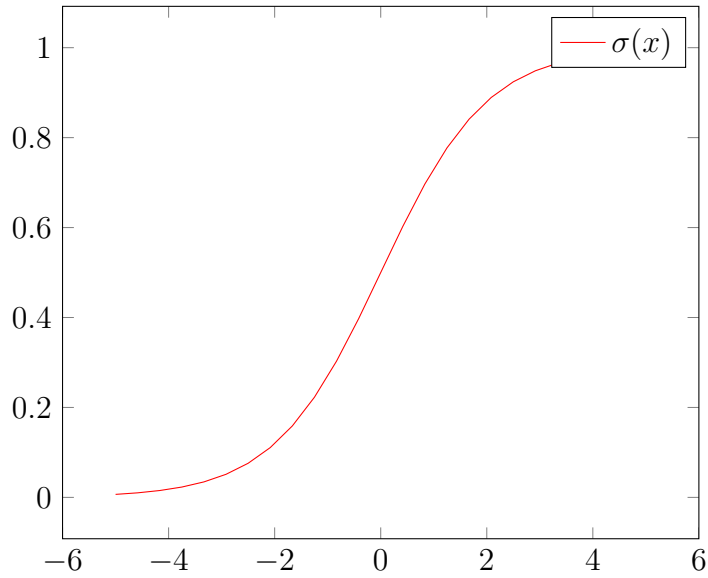
Classical Machine Learning

4.1 Support Vector Machine (SVM)

so far the cost function with regularization is

$$\frac{1}{m} \left[\sum_{i=1}^m y^{(i)} (-\log(h_{\theta}(x^{(i)}))) + (1 - y^{(i)}) (-\log(1 - h_{\theta}(x^{(i)}))) \right] + \frac{\lambda}{2m} \sum_{j=1}^n \theta_j^2$$

a historical, and widely used alternative yet similar algorithm is support vector machine or SVM, but first let's delve deeper inside the sigmoid function.

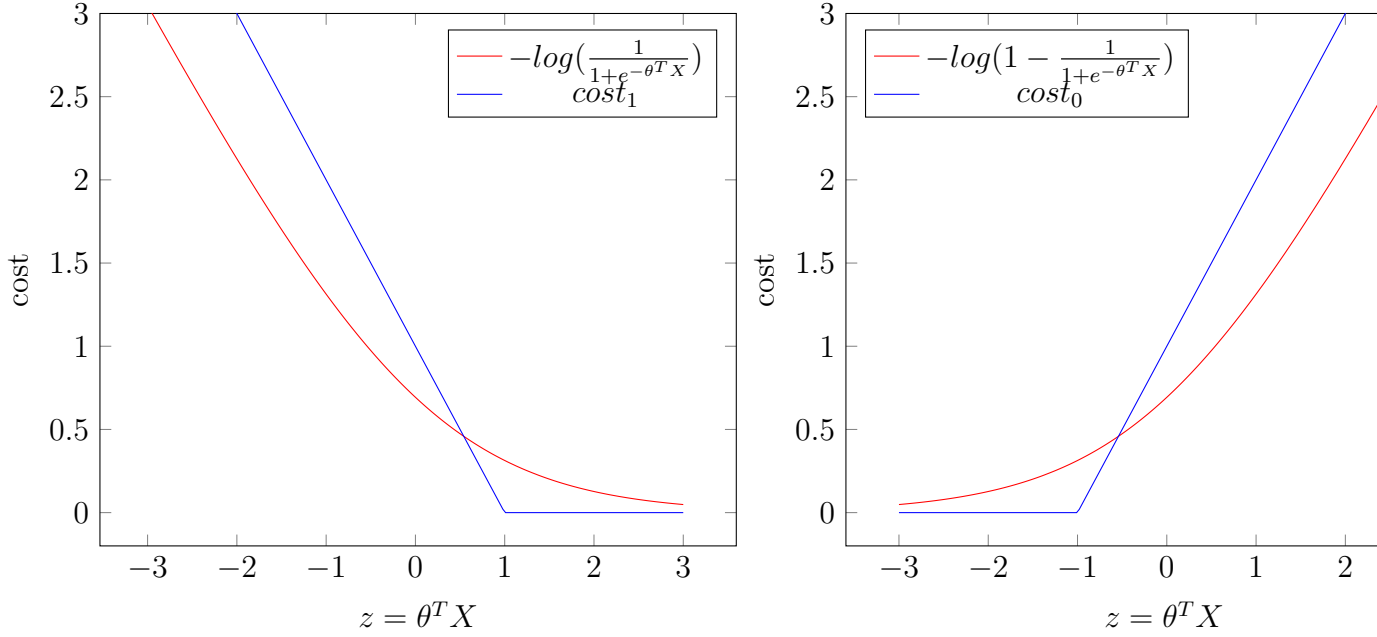


to get $y = 1$ when $z \gg 0$, on the other end for $y = 0$ then $z \ll 0$.

if we generalize our look to the logistic cost function, if $y = 0$ then the equation reduces to

$$\frac{1}{m} \left[\sum_{i=1}^m y^{(i)} (-\log(h_{\theta}(x^{(i)}))) \right] + \frac{\lambda}{2m} \sum_{j=1}^n \theta_j^2$$

, we cant define a define $cost_1 = -\log(h_{\theta}(x^{(i)})) = -\log(\theta^T X)$



$cost_1$: the linear lower bound of $-\log((h_\theta)(x^{(i)}))$ term, when $y=1$, and $\theta^T X \geq 1$

$cost_0$: the linear lower bound of $-\log(1 - h_\theta(x^{(i)}))$ term, when $y=0$, and $\theta^T X \leq -1$

penalization term λ is added to the first term instead, $C \approx \frac{1}{\lambda}$, and finally, and averaging by m can be ignored since we multiply by large value C , and can also be ignored from the regularization term, since the θ is adjusted in the first term, note that the regularization term is just the Euclidean norm $\|\theta\|_2$

the final SVM:

$$C \sum_{i=1}^m [y^{(i)}(cost_1(\theta^T X^{(i)})) + (1 - y^{(i)})(cost_0(\theta^T X^{(i)}))] + \frac{1}{2}\|\theta\|_2$$

and the intuition for this is the first term get trained such that when $y = 1$ then $cost_1 = 0$, if $y = 0$, then $cost_0 = 0$, thus the first term is set to zero, and then SVM shrinks down to minimizing

$$\frac{1}{2}\|\theta\|_2$$

4.1.1 Large Margins in linear Decision Boundaries

it turns out that SVM do a pretty good job in decision boundary for given data-set, assume we classify class A against class B in the given data-set, first in a linear classification, it draw a line in between the two blobs of data, with nearly equal margins in both sides, the reason is in the hypothesis constrains $z = \theta^T X$, for $y = 1$ then $z \geq 1$, and if $y = 0$ then $z \leq -1$, thus the model is trained to classify classes A,B outside the Large margin $[-1, 1]$, also note that the $\theta^T x^i$ is the dot product (degree of alignment) between the weight parameter, and the training example x^i , and since z is required to be outside the defined range, thus z need to be a large value, meaning that both $\vec{x}$, $\vec{\theta}$ need to be aligned, otherwise z shrinks to zero at perpendicular angle 90° . if for example $\vec{\theta}$ is inclined w.r.t $\vec{x}^i$ by Θ° , then $\vec{\theta}$ need to be pretty large in order for z to lay outside $[-1, 1]$, but since we using penalizing factor C , then $\vec{\theta}$ is maintained to reasonable value, and thus the SVM model converges to the optimal decision boundary with large margins.

4.1.2 Non-linear Decision Boundaries

previous hypothesis $\theta^T X$ is linear, in case of non-linear features, for example extracting faces for pictures, the problem is no longer classification of two categories, but rather features extraction of non-linear features from n-Dimensional example of n features. for example $f(x) = \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_1^2 + \theta_4 x_2^2 + \theta_5 x_1 x_2 + \theta_6 x_1^2 x_2^2 + \dots$ can be a good representative for capturing the non-linear boundaries for training example x^i for all i in n . for a problem with large number of features is equation can be very expensive to train, instead choosing a appropriate polynomial of k -degree, to suits our problem decision boundary can be easier to train.

4.1.3 Gaussian kernel

in a non linear boundaries, where class A is surrounded by the class B scattered plots, the most appropriate used function is the normal, or Gaussian distribution function

$$f(x) = \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

yet instead of μ we use locality points (we will discuss this in the convolution neural network chapter), and since we deal with vector the numerator is the Euclidean distance $\|x^{(i)} - l^{(i)}\|$.

so for each training example $x^{(i)}$ if it exists in the neighbourhood of the locality point $l^{(j)}$ for $i, j \in m$. with computational cost $O(m^2)$ then $f(x)=0$, otherwise if the distance is too far, $f(x)=1$, thus SVM with Gaussian kernel hypothesis:

$$C \sum_{i=1}^m [y^{(i)}(cost_1(\theta^T f^{(i)})) + (1 - y^{(i)})(cost_0(\theta^T f^{(i)}))] + \frac{1}{2} \|\theta\|_2$$

4.2 Unsupervised Learning (Introduction)

in **supervised learning**, given structure data-set of input X, and output Y, and the model has to be trained to map $X \rightarrow Y$, in **unsupervised learning**, given only input X, the model has to be trained to extract similarities between input examples, thus classifying them into different coherent classes, for example in NLP, using clustering k-mean algorithm to compute word embedding similarity, or another more interesting example is identification of galaxies, and stars from astronomical pictures, another example is market segmentation, even when the data doesn't fit into separate coherent clusters, cut extend over a long range, it can be divided onto K number of classes/clusters

4.2.1 k-mean algorithm

given and input $X = x^1, x^2, \dots, x^m$, randomly sample the K centroids (cluster centroids) over X, secondly is the assignment step,

in which for each example x^i for all $i \in m$, $c^{(i)}$ is assigned to the centroid with the shortest distance, thus taking the values $[1-k]$, so if x^i is closest to k^{th} then $c^{(i)} = k$, thirdly, and final step is the centroid movement, in which for each centroid μ_k for $k \in K$, is relocated to the average of all points assigned to it.

the cost function

$$J = 1/m \sum_{i=0}^m \|x^{(i)} - \mu^{(i)}\|^2$$

to avoid falling into a local optima, multiple instantiation of the algorithm with different random initialization is advised, at the end the lowest cost is chosen.

finally, if the number of clusters is unknown, and set arbitrary, then it need to commit to the constraint that $K \leq m$, and whenever a centroid isn't assigned with any example, then K can be reduced by this number of unassigned centroids, or number of clusters can be plotted against the cost function, and we choose the pivotal k where the slop of cost changes significantly if any exist called **Elbow method**.

4.2.2 Principle Component Analysis PCA

another technique that will come useful later on in the NLP chapter is dimensionality reduction, having high-dimensional input dataset, with potentially redundant features, reducing the dimensions can be useful for both understanding data, and getting rid of redundancy, so for training data set with m training examples, of n features, let's assume that the input $X \in R^n$ is projected into smaller subspace R^k , ignoring the bias feature term for simplicity.

a straight forward algorithm is the Principle component Analysis, or PCA, for input features $X_{m \times n}$, $\Sigma = X^T X$ is the input for the single value decomposition SVD algorithm to de-factor it into $U \Sigma V$, for U being the Eigen vectors representing the n Dimensional space, and Σ is the Eigen values multiplied by I_n , in which the orthogonal matrices U , V have the property that any subset of U column for example first k columns denoted U_k satisfy

$U_k^T U_k = I_k$, therefore the projection into the k-Dimensional subspace spanning X is $U_k X$, or the projection of X on the subspace U_k .

reduction shouldn't be done without losing much of the useful data, so it's wise to pick k , such that the lose of variance $<$ negligible value ϵ , and can be estimated by the sum of the reduced eigen values, over the total sum in n dimension, so as to say

$$\frac{\sum_{i=1}^k S_{ii}}{\sum_{i=1}^n S_{ii}} \geq 1 - \epsilon$$

.

Chapter 5

Structuring machine learning

5.1 Machine learning strategy

we discussed in previous chapter how, and when to use optimization, regularization techniques, in practice, such decision making is crucial, and can save months of development time if we know to choose the right direction, and make the right decision to help our model work more effectively, doing regularization when the system need optimization in the case of high-bias is time consuming process, and spending months getting more data-set in highly biased model can be a total waste of time.

5.2 orthogonality

tuning process isn't done arbitrarily but we must know which hyper-parameters to tune to get the most desired effect.

modeling a process or a problem with as many features, we need to be able to fit our model to capture all data-set features. in the set of hyper-parameters it's easier if we can tune one parameter from the set of hyper-parameters and only take an effect on the corresponding purpose without avalanche effect, so for example in a car we can choose the adjust speed separately from the steering, although steering and speed are related in a single function, yet we can change one without affecting the other, therefor the name **orthogonality**

doing well on training set, usually comparable to human level performance, depending on the application,... change the network into different architecture, use more efficient optimization algorithm i.e ADAM, etc

doing well on the dev-set, in that order if the model is not fitting the dev set, meaning we have a high-variance, and we can use regularization, or getting a larger data-set.

fit test-set well on the cost function, if we can't get our model to fit on the test-set in the other, then we probably over-tuned the model on the dev-set, and we need to re-tune with bigger dev-set.

perform well in application, finally and in this order, if we can't capture the real application features in our model, then we then dev-test sets aren't quite fitting in the model, or we need to change the cost function.

5.3 Set up your Goal

5.3.1 Evaluation metric

5.3.2 Precision vs accuracy

accuracy/recall numerically the average of our prediction measurement is how far close to the reality.

on the other hand, precision is how far our network prediction are True, or in other words is how far our predication are what they are.

precision, and accuracy are orthogonal metrics, but there is a trade-off between both of them, in application, in the training process, it's advised to train different simultaneously (if possible) and plot their precision/accuracy metrics, and choose the highest of all, getting high precision in one model, or high accuracy on the other, can't indicate which model is better.

5.3.3 F1-score

this metric is harmonic mean of precision $\mathbf{p}$, and accuracy/recall $\mathbf{R}$:

$$\frac{2}{\frac{1}{P} + \frac{1}{R}}$$

with a single metric that combine how far our model is both accurate, and precise can be more efficient.

5.3.4 Satisfying-Optimizing metric

we might have a precise, and accurate model such as in case of F1-score, but yet it doesn't satisfy our practical application, in reality we are limited by time, and computational resources, and we need high **F1-score** relative to available resources, then a more general metric is a weighted average of F1-score, and our resource metrics, for example if we developing a network for recognition on phone we need the system to be both accurate/precise, but also to be under specific time threshold T , then our evaluation metric can be $F1 - score * 0.8 + T * 0.2$.

5.4 train/dev/test sets

as discussed earlier in previous chapter that the ration of data-set partition into train, dev, and test sets are dependent on the data-set size, and the application really in accord with bias/variance scale.

having a large enough training set can reduce variance, and sufficiently large enough dev set can enable us to have a deep perspective into the bias-variance scale, and finally the test set is essential to evaluate how var our low-bias, low-variance model fits on the same distribution of data.

5.4.1 dev/test metric

in case of very domain-specific training algorithm, for example a classification network on a very specific class c_i or object out of the set of different classes C from our data-set of size n , we need to train our model to be aware of different classes, well distinguish between object rather than cat/not-cat binary classification, but also be able to classify our target object efficiently enough, we can add ad weighting parameter to the cost function evaluation:

$$J(w, b) = \frac{1}{\sum_{k=1}^{C_k} w_k} \sum_{i=1}^m w_k L(\hat{y}, y)$$

$$\text{where } w_k \text{ is the weight corresponding to each class } \begin{cases} w_1 \leftarrow \hat{y} = c_1 \\ w_2 \leftarrow \hat{y} = c_2 \\ \dots \\ w_n \leftarrow \hat{y} = c_n \end{cases}$$

it's important to note that if the distribution of the practical input-set is of different features than that of our train/dev/test distribution, our model can be guaranteed to do well in practice.

5.5 Human-level performance

in the past few years machine learning have surpassed human cognitive levels in different areas, performance of machine today overshadow that of humans, in the past decades the accuracy of machine has been increasing linearly up to level slightly higher than the human's.

as long as the machine level is lower from that of the humans we can boost it with human-labeled data, gain a better insight from error analysis, better analysis of bias/variance , as to increase it's cognitive abilities

5.5.1 human(bayes) error

as machine level rises high to humans, we can add another level the train/dev/test cycle, that is human level error, and interpretation from human/bayes level to training error is called the **avoidable bias** error,

and difference in error between training and development is the **variance**.

we can use avoidable bias, and variance as a metric for how well our model perform, and optimize our model, and network to reach human level performance, and minimize that error.

comparing those two metrics tell us where to optimize, for example if the avoidable bias is 4

the human level performance is really lose, comparable to how well our model is trained to achieve low variance error, humans as well difference in their levels of cognitive ability fitting a normal distribution naturally, but if an average human is to be compared to a guru in the specific domain, then the guru of sufficient skill-set, and training is able to achieve lower Bayes error, and the latter varies greatly, for example in radiology, an average human is expected to achieve higher error of diagnosis, and recognition compared to skilled doctor, and the latter compared to a team of doctors, choosing specific human-level from the domain of human-cognition is another problem.

5.6 Error analysis

taking the right decisions in what to invest your time in while optimizing a model can be a great leverage and can save month, if not years of Engineering time! making error analysis is just one of those.

statistical analysis of the mismatched predictions $\hat{y}$ at the test time cat is coined the name **Error analysis**.

Assume for example that error analysis taking from the sample of mismatched predictions shows that false-negative is 50% percent, then it would be wise to spend time enhancing our model for predictions, and cut down the ratio of 50% to a reasonable ratio, perhaps 2% .

for example in a classifier network, out of N mismatched predictions are actually what we expect (of being a specific class), but the model classified otherwise, from the N mismatched predictions we take for example 100 entry, and analyse the false-negatives. if it shows that the ratio is 50%

for example then we are on the right track to regard this as bias problem, and invest time on it, on the other-hand if out of the 100 entries only 3% were false-negatives then, it's not our priority to invest time down this road, and to achieve such high accuracy is actually indication that we might not be able to do better, and it's more wise to move on, and spend time optimizing our model to get a low false-negatives on other classes from our error analysis, or train our model on certain features that might be mis-classified.

5.6.1 Mislabeled data

another perspective to Error analysis is to record the mislabeled input data, it turns out that deep learning is quite robust against small randomly mislabeled data, so in big data of millions of entries, if the mislabeled data are randomly distributed over the data-set classes, and it's a below acceptable error, it's really unwise to spend months fixed mislabeled data since deep learning prove to be robust against just errors.

so far there are three metrics in error analysis to consider, first the overall false-negatives, we take this ration, and sum the ratio of each prediction, thirdly the ratio of mislabeled data, comparing the three metrics can give us insight to help enhance our model, and reduce the overall error, if the overall error is reduced, meaning our model accuracy is acceptable compared to human level, we still have two metrics, to choose from, if the prediction-error (overall-error - mislabeled-error) from the dev-set is reduced after we choose between models, and the test error is also reduced then it can be wise to spend time on mislabeled input data.

5.7 Mismatched training dev/test sets

it turns out in practice the model's input usually tend to noisier than the training/dev/test sets input, which are filtered for example in computer vision, input data-set can be of a common distribution of high-resolution, close proximity, centered object, while user input can be blurry, of low-resolution, or in speech recognition we train

our model on a clear audio, yet in practice there will be stutter, background noise, etc.

there is a trade-off between choosing data-input between different distribution, and accuracy, and choosing data from different distribution can really help but under the following criteria, that the dev/test phases need to aim at our target, meaning to be chosen completely from target data, while the training data-set can be a shuffle of different distributions.

in this scenario new error is introduces, consider that the training error is 1% and the dev error is 10% then it's possible to tell wither our model is over-fitting or it's result of mismatch data from different distribution?! to help make a decision, the data-set is divided into training/train-dev/dev/test sets, where train-dev, and training sets are form the same shuffled distributions, while dev/test sets are of our target data, so addressing the same problem if the train error is 1%, and the train-dev error is 9.5%, while the dev error is 10% then it's a variance problem, but if the train-dev error is 1.5%, and the development error is 10% then our model is getting mismatched data.

lastly, and as previously discussed if the test error say 15% then our test set of the same target distribution as the dev-set is over-fitting the dev-set, and to fix this, we ought to increase the dev-set.

5.7.1 Addressing data mismatch

in case of data mismatch in a shuffle of distributions, or in other words the wide error difference between train-dev set, and the dev set, this is an indication of the gap between the data-sets from different distributions.

a close examination of that difference through error analysis help taking the right decision to reduce this difference, and choosing training data from distribution closer to the target data.

5.7.2 Artificial data synthesis

if the target data-set is limited as usual in practice, given a target data set of size m , it's possible through artificial data synthesis

techniques to increase this size to 100m, or even as large as 1000m, but in this case it's possible to run into an over-fitting problem if the original m data set isn't representative enough.

5.8 Multiple tasks learning

5.8.1 Transfer learning

when we train a model in a very domain-specific application with very limited data-set, we can employ weight-transfer, also known as transfer learning, and pick the closest general domain to our domains-specific application, and randomly initialize the last layer's weight $w^{[L]}$, leaving the rest of weight as they are, and through the new domain-specific data set (X, Y) old $w^{[l]} \mid l \in [0 - L[$, and randomly initialized $w^{[L]}$ we can scale the old model to the new model very quickly.

5.8.2 Multitask learning

in transfer learning we make use of previously trained weights to train our model on a newly different task usually in cases when there isn't sufficient data of the second class, but in cases when there is a need to extract as many features from the same example, and the same model, previously we discussed softmax activation function for training a model for multi-class classification, but in different cases of multitasking models rather than multi-class model, for example in object detection in a video, we need to extract boundaries around n set of object $\{o_1, o_2, \dots, o_n\}$ with bounding box surrounding each object, compared to multi-class problem then we just satisfied with the prediction of $\hat{y}$ in a sparse hot-vector, but in the multitask the label Y is vector with more than a single entry of each task, in this we employ what is called Multitask learning, in the following chapter we will deal such case.

5.9 End-to-End Machine Learning

Chapter 6

Computer Vision

6.1 Object Detection

In previous chapters we discussed classification neural networks, for a given input x_i , being an image, and prediction (estimate) y is the classification wither in a binary representation 0/1 of size 1, or on-hot vector of multi-class classification, in case the input contains more than one class, known as multi-tasking. so far

classification in computer vision is one of the building blocks, feeding input x_i to the NN, and getting an estimate of the classes inside the pictures isn't sufficient, when we (humans!) look at a picture we first locate the objects, then classify each of them, then derive a meaning. in computer vision we focus on the first two steps.

object detection is about How to locate all the objects in the picture simultaneously? but before that, in previous chapter we discussed that it's advised to use min-batch in the case of large NN compared to the available computational power, and memory, having a moderate resolution picture as an input i.e 1024×1024 of 3 channels, if the first layer consist of just 1000 neurons we end up with billions of parameters to train! with huge amount of parameters, and features to capture, if we doesn't have sufficient data-set, then our model will be over-fitting the data-set, then how to solve it?

convolution solves this problem, it makes use of the **sparsity of connections** (in each layer, each output value depends only on a small number

of inputs), and **parameter sharing** (that a single features detector or a filter is shared between features) and we will discuss it in the following sections.

due to sparsity of connection convolutions NN good at capturing translation invariance, meaning a network is capable of getting the same output of a slightly shifted image, we will see why.

6.1.1 Edge Detection

for input image x_i how to detect the boundaries? how to interpret it mathematically?

first, edge boundaries of 2×2 , or 4×4 pixels can be vertical, horizontal, or inclined by n° , to extract such features of image $g(x)$ by filter $f(y)$, is a process of weighting image $\mathbf{g}$ by the filter $\mathbf{f}$, or mathematically, it's the integral of all the shifts in shifted, and reversed function g , or f

$$\int_{-\infty}^{\infty} f(\tau) * g(x - \tau) d\tau$$

(see the appendix on convolutions)

to detect a vertical edges a vertical edge detection can be as simple

as $\begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix}$, or with more emphases on the center.

for example to detect horizontal edges, we employ $f(\tau) = \begin{bmatrix} 1 & 1 & 1 \\ 0 & 0 & 0 \\ -1 & -1 & -1 \end{bmatrix}$,

for image $g(x) = \begin{bmatrix} 10 & 10 & 10 & 0 & 0 & 0 \\ 10 & 10 & 10 & 0 & 0 & 0 \\ 10 & 10 & 10 & 0 & 0 & 0 \\ 0 & 0 & 0 & 10 & 10 & 10 \\ 0 & 0 & 0 & 10 & 10 & 10 \\ 0 & 0 & 0 & 10 & 10 & 10 \end{bmatrix}$ then convoluted $\int_{-\infty}^{\infty} f(\tau) * g(x - \tau) d\tau$

$$\text{evaluated to } \begin{bmatrix} 0 & 0 & 0 & 0 \\ 30 & 10 & -10 & -30 \\ 30 & 10 & -10 & -30 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

more filter examples:

- **sobel's filter** is $\begin{bmatrix} 1 & 0 & -1 \\ 2 & 0 & -2 \\ 1 & 0 & -1 \end{bmatrix}$.
- identity filter function $\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$
- another sharper edge detection filter $\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$
- sharpen filter $\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$
- blur box filter $\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$
- Gaussian blur filter $\frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$
- larger 5×5 Gaussian blur filter $\frac{1}{256} \begin{bmatrix} 1 & 4 & 6 & 4 & 1 \\ 4 & 16 & 24 & 16 & 4 \\ 6 & 24 & 36 & 24 & 6 \\ 4 & 16 & 24 & 16 & 4 \\ 1 & 4 & 6 & 4 & 1 \end{bmatrix}$.

6.1.2 Padding

from previous example you can notice that 6×6 input image was shrunk into 4×4 matrix (**known as valid convolution**), also the boundary edges are dismissed, to maintain the same output size to the input size (**known as the same convolution**) we use padding:

for input image of size $n \times n$, and filter of size $f \times f$, the output is $n - f + 1 \times n - f + 1$, for same convolution we pad the edges with p cells, such that $p = f - 1/2$ then final output image size $n + 2p - f + 1 \times n + 2p - f + 1$, for this equation to hold, then filter size have to be **odd**, and it's usually the case in computer vision, look at the **identity filter**, where the weighting parameters are set to zero, while the **center pixel** is set to 1.

to pad our previous examples we get the following image with $p = 1$

$$g(x) = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 10 & 10 & 10 & 0 & 0 & 0 & 0 \\ 0 & 10 & 10 & 10 & 0 & 0 & 0 & 0 \\ 0 & 10 & 10 & 10 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 10 & 10 & 10 & 0 \\ 0 & 0 & 0 & 0 & 10 & 10 & 10 & 0 \\ 0 & 0 & 0 & 0 & 10 & 10 & 10 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

6.1.3 striding

?

6.1.4 Convolution on RGB channels

In 3 RGB channels $n_c = 3$, on image $n \times n$ we add filter for each channel $f \times f \times n_c$, and the output image is $n - f + 1 \times n - f + 1 \times 1$

6.1.5 Multiple filters Convolution

for layer L of size $n \times n \times n_c$, there is always more than a single features, and therefore requires filter for each feature, so filtering layer end up being $f \times f \times n_c \times n^*$ such that n^* is the number of filters for layers, and the output is $n - f + 1 \times n - f + 1 \times n^*$.

6.1.6 Example ConvNet

in case of input image $n_H^{[0]} = n_W^{[0]} = 39, n_c^{[0]} = 3$ of size $39 \times 39 \times 3$, using 10 filter of size $f^{[1]} = 3$, and strides $S^{[1]} = 1$, padding

$P^{[1]} = 0$ and 10 filters, then layer 1 will deterministically be of a size $\left\lfloor \frac{n^{[1]} + 2p^{[1]} - f^{[1]}}{s^{[1]}} \right\rfloor + 1 = 37$, then $n_H^{[1]} = n_W^{[1]} = 37, n_c^{[1]} = 10$.

in layer 2, using 20 filter of size $f^{[2]} = 5$, and strides $S^{[2]} = 2$, padding $P^{[2]} = 0$, then layer 2 will deterministically be of a size $\left\lfloor \frac{n^{[2]} + 2p^{[2]} - f^{[2]}}{s^{[2]}} \right\rfloor + 1 = 17$, then $n_H^{[1]} = n_W^{[1]} = 17, n_c^{[1]} = 20$.

in layer 3, using 40 filters, of size of size $f^{[3]} = 5$, and strides $S^{[3]} = 2$, padding $P^{[3]} = 0$, then layer 3 will deterministically be of a size $\left\lfloor \frac{n^{[3]} + 2p^{[3]} - f^{[3]}}{s^{[3]}} \right\rfloor + 1 = 7$, then $n_H^{[1]} = n_W^{[1]} = 7, n_c^{[1]} = 40$.

flatten layer3 $7 \times 7 \times 40$ into 1960×1 and activate it through softmax/logistic function, where last layer 5 is 1×1 .

6.1.7 Pooling Convolutions

along side convolution NN, there Pooling layers are used to make the network more robust, and cut down the size of parameters.

most used parameters are $f = 2, s = 2$, or $f = 3, s = 2$, and padding is rarely used.

note that there is no learned parameters in pooling, so max-pooling layer doesn't count as a stand alone layer since no parameters to train, and rather seen as part of the layer operating on.

6.1.8 Max Pooling

pooling is $n \times n$ layer, and services to divide the input layer into smaller $n \times n$ layers and leaving only the maximum value of which it's an **argmax** convolution function, the convolution function takes the argmax instead of the integral.

for example, input layer $\begin{bmatrix} 1 & 3 & 2 & 1 \\ 2 & 9 & 1 & 1 \\ 1 & 3 & 2 & 3 \\ 5 & 6 & 1 & 2 \end{bmatrix}$, with $strides = 2$, convoluted by 2×2 max-pooling into $\begin{bmatrix} 9 & 2 \\ 6 & 3 \end{bmatrix}$

6.1.9 Average Pooling

similar to max pooling except argmax is replaced with the mean function.

for example, input layer $\begin{bmatrix} 1 & 3 & 2 & 1 \\ 2 & 9 & 1 & 1 \\ 1 & 3 & 2 & 3 \\ 5 & 6 & 1 & 2 \end{bmatrix}$, with $\text{strides} = 2$, convoluted by 2×2 , max-pooling into $\begin{bmatrix} 3.75 & 1.25 \\ 4 & 2 \end{bmatrix}$.

6.2 Examples

6.2.1 LeNet-5 Network

1. Layer 0: with input image $n_H^{[0]} = n_W^{[0]} = 32$, $n_c^{[0]} = 3$.
2. Layer 1: using 8 filters, $f^{[1]} = 5$, $s^{[1]} = 1$, we get $28 \times 28 \times 8$, adding max-pooling with $f = 2$, $s = 2$, pours down into $14 \times 14 \times 8$ layer.
3. Layer 2: using 16 filters $f^{[2]} = 5$, $s^{[2]} = 1$, we get $10 \times 10 \times 16$, adding max-pooling with $f = 2$, $s = 2$, pours down into $5 \times 5 \times 16$ layer, flattened into $400 \times 1 \text{vector}$
4. Layer 3: Fully-Connected (FC3) layer of 120 neurons, with weight, bias parameters $W^{[3]}, b^{[3]}$.
5. Layer 4: FC4 layer of 84 neurons, and $W^{[4]}, b^{[4]}$, feed to softmax activation with 10 neurons output.

notice the size of parameters in convolution layers compared tota **FC** layers, in **CONV1**, **CONV2** number of parameters are 208, 416 respectively compared to 48001, 10,081 in **FC3**, **FC4** respectively. meanwhile the **Activation size** falls down gradually, from 3.072 the number of pixels in the image to 10 neurons in the softmax classification, and 6.272, 1.568, 1.600, 400, 120, 84 in CONV1, POOL1, CONV2, POOL2, FC3, FC4 respectively.

6.2.2 AlexNet Network

1. Layer 0: input is $227 \times 227 \times 3$,

2. Layer 1: applying 96 filter with size 11, and stride=4, the output is $5 \times 5 \times 96$ max pooled by 3×3 of strides of size 2, the input is $27 \times 27 \times 96$.
3. Layer 2: applying 256 filter, of **SAME** padding, of size 5, outputs $27 \times 27 \times 256$, max-pooled by size 3, and stride size=2 outputs $13 \times 13 \times 256$.
4. Layer 3: 384 filters with SAME padding convolutions, outputs $13 \times 13 \times 384$.
5. Layer 4: 256 filters with SAME padding, outputs $13 \times 13 \times 256$ max-pooled by size 3, and strides of size 2, outputs $6 \times 6 \times 256$ flattened into 9216×1 .
6. Layer 5: fully-connected 4096×1 , by 4096×1 , then softmax of 1000 neurons/predictions.
7. compare the two networks, LeNet-5 from the nineties with around 60000 parameter, to 2012 AlexNet with 60 million parameters, despite this, they are similar in structure.

6.2.3 VGG-16 Network

VGG similar AlexNet, yet deeper, with narrower convolution proven to be more powerful.

using [CONV] of size 3×3 , strides size=1, of SAME padding, MAX-POOLing of size 2×2 , with strides size=2.

consist of 16 layers therefore the name VGG-16, with 138Million of parameters, and it's network is: $[CONV64 \times 2] \rightarrow [CONV256 \times 3] \rightarrow [CONV512 \times 3] \rightarrow [CONV512 \times 3] \rightarrow [FC4096] \rightarrow [FC4096] \rightarrow [SOFTMAX1000]$

1. $224 \times 224 \times 3$ input image.
2. applying $[CONV64] \times 2$, (convolutions of 64 filters twice), outputs $224 \times 224 \times 3$. [MAX-POOL]ed into $112 \times 112 \times 64$
3. $[CONV128] \times 2$ [MAX-POOL]ed into $56 \times 56 \times 128$.
4. $[CONV256] \times 3$ [MAX-POOL]ed into $28 \times 28 \times 256$

5. $[CONV512] \times 3$ [MAX-POOL]ed into $28 \times 28 \times 512$
6. $[CONV512] \times 3$ [MAX-POOL]ed into $7 \times 7 \times 512$
7. $[FC4096]$ fully connected layer with 4096 nodes
8. $[FC4096]$ fully connected layer with 4096 nodes
9. $[SOFTMAX1000]$ softmax activation with 1000 entries.

6.2.4 ResNet (Residual Block)

Very deep network are much difficult to train because of vanishing, and exploding gradients, to make this feasible, **Residual blocks** was introduced

even with the use of an Optimizer to lessen the vanishing/exploding gradients, in a very deep network with hundreds or even thousands of layers, which are rarely used, the number adds up, and either diminishes to zero, or explode into large numbers. residual blocks adds the activation from a previous layer (i.e $l - 2$) into the current activation function l , $a^{[l]} = g(z^{[l]} + a^{[l-2]})$, also known as **shortcut**, or **skip connection**.

assume for example we using regularization and $w^{[l]}, b^{[l]} \rightarrow 0$, then $a^{[l]} = g(a^{[l-2]})$, residual would prevent vanishing gradient.

note that skip connection between $a^{[i+n]}$, and $a^{[i]}$ for $i, n \in N$, can be between two different sizes, in practice same padding is used between shorted layers, or through function W , such that shape of $a^{[i+n]} = W.a^{[i]}$. that can for example zero pad the i th layer.

6.3 Inception

another parameter to worry about is the size of convolution kernel, instead of testing out different sizes, it's possible to combine all of them at once with SAME padding, and let the model train, and figure out the most convenient of all, called **Inception network**, on a computational cost.

for example in input network of size $28 \times 28 \times 192$, we can combine 1×1 , 3×3 SAME padding, 5×5 SAME padding, and MAX-POOLing SAME padding, to get $28 \times 28 \times 256$ network. in this example note that the computational cost is for 120 million parameter.

6.3.1 1×1 Convolutions

in such very expensive layer, what we can do to reduce the computational cost is convolution of 1×1 times the factor or reduction, for example in the previous example we can pick fittest filter through [CONV1], with 16 kernel, applying second [CONV5] with 32 filters we end up with $28 \times 28 \times 32$ layer, so we manage to pick the fittest kernel, and also reduce the computation cost for 120 million parameter to just 12.4 million.

6.3.2 Inception Block

the inception network is a series of block of the following building block: (insert the szegedy paper picture here)

6.3.3 Inception Branches

(insert the szegedy paper picture here)

6.4 Object Detection

6.4.1 Localization and Detection

in computer vision network the output layer is usually a vector of localized objects rather than an one-hot vector representing a specific class, and the process is divided into three steps, first **Detection**, second **Localization**, lastly **classification**.

6.4.2 classification with localization

one-hot vector SOFTMAX output can be combined with localization, to locate, specific target class in an image, boxing our target object with $b_w \times b_h$ with Cartesian coordinates of the image center (b_x, b_y) , now three are 4 more parameters. b_x, b_y, b_w, b_h for localization.

labeled output vector with localization:

$$\begin{array}{|c|} \hline p_c | \text{a Boolean for wither there is an object or not} \\ \hline b_x \\ b_y \\ b_h \\ b_w \\ c_1 | \text{first class} \\ c_2 | \text{class 2} \\ \dots \\ c_n | \text{class n} \\ \hline \end{array}$$

. if $p_c = 0$ then the rest of the vector's elements will be zero.

Loss function $L(\hat{y}, y)$ will be element wise operation. but practically different loss functions can be used on different ranges of y for example log-likelihood on range $[c_1 - c_n]$, and mean squared on the bounding box coordinates range, and logistic regression loss.

6.4.3 Landmark detection

it's possible to go deeper beyond the bounding box, and train model on labeled data-set with desired landmarks on the target object, for example after along side localization of the humane face with bounding box, it's possible to add landmark features for example to locate the human mouth, eye's, and cheek boundaries, so to extract the facial expressions, another example could be tracking the skeleton, and joints for pose estimation of an athlete during exercise, also used in augmented reality applications.

to place 50 landmark on a single object to locate it's boundaries with two Cartesian coordinates points means extra 100 entry inside the labeled data-set entries.

6.4.4 Object Detection

to detect, locate objects in input image of size $N \times N$.

run sliding window cropping function over input image cropping images with re-sizable starting with length = 1, with certain strides.

sequentially iterate through the previous step increasing the length 1 for larger crops, this is a problem with $O(N^2)$ computational complexity.

then final step is to run the cropped images in the classifier.

6.4.5 Convolutional sliding windows

sliding window cropping is computationally expensive, if you have noticed that for every crop, the model expected to forward the cropped image to be classified, then increasing the complexity by the number of crops.

the shared parameters characteristic of the convolutions can a great leverage to solve this problem, and cut the complexity significantly, by running the whole iterations simultaneously, it can be illustrated by an example.

6.4.6 Example:

for the following network classifying input image(expected input) of size $14 \times 14 \times 3$, $[CONV5] \rightarrow [MAX-POOL2-5] \rightarrow [FC5-400] \rightarrow [FC1-400] \rightarrow [SOFT-MAX-4]$. with the corresponding size $10 \times 10 \times 16 \rightarrow 5 \times 5 \times 16 \rightarrow 1 \times 1 \times 400 \rightarrow 1 \times 1 \times 400 \rightarrow 1 \times 1 \times 4$ we end up with 4 neurons (softmax output) being our classification model, for an input image with size $28 \times 28 \times 3$ and sliding window 14×14 to fit in our model, running the same convolution network above on this input $[CONV5] \rightarrow [MAX-POOL2-5] \rightarrow [FC5-400] \rightarrow [FC1-400] \rightarrow [SOFT-MAX-4]$. with the corresponding size $24 \times 24 \times 16 \rightarrow 12 \times 12 \times 16 \rightarrow 8 \times 8 \times 400 \rightarrow 8 \times 8 \times 400 \rightarrow 8 \times 8 \times 4$ such that the most upper right cell $1 \times 1 \times 4$ of coordinates (0,0) is actually corresponding to the simplified model output above, and

the second (0,1) output is one slide to the right, and so on.

6.5 Bounding Box prediction

the aspect ratio of the bounding box isn't really fixed, and to wrap the detected object correctly, then there are two variables in the sliding box instead of one in sequential terms.

6.5.1 YOLO

YOLO is a convolutional network, but we can explain it from a sequential iterative perspective that the input image is divided up into $k \times k$ cells, running the sliding window function in each cell, then simultaneously, so instead of getting a single 8×1 output, we get $k \times k \times 8$ output, the localization algorithms runs in a single cell.

even if the object stretches on multiple grid cells, it's only assigned to a single cell.

to specify the bounding box inside a grid, the center of the detected object is coordinated with respect to the origin(0,0) of the cell on the top most left, and the bottom right assigned to (1,1), therefore the center(b_x, b_y) must be between 0,1, where as the b_w, b_h can be greater than 1 if it wraps multiple cells, and it's measured by the ratio with respect to the cell width, or height respectively.

6.5.2 Jaccard distance, or Intersection over Union IOU function

it's an accuracy metric for how well the model is performing, and defined by the area of intersection between the prediction, and the labeled bounding boxes, over the prediction area. indicating how far this predicted bounding box fits into the ground truth table.

6.5.3 Non-Max Suppression

in practice the input image is divided up into very fine cells, and a single object can overlap multiple cells, Non-Max suppression is algorithm to choose the maximum overlapping cell, and assign the detection with is, suppressing other cells, therefore the name.

the metric for overlap is the p_c in the output layer, previously we assigned p_c to either 0, or 1, but in this case is the probability of overlap, and only the highest bounding box's p_c remains, suppressing the rest.

6.5.4 Anchor Boxes

in a single cell there could coexist more than one object, we expect multiple object each with specific aspect ratio, then we consider more than sliding window also known as **Anchor Box**, each with different aspect ratio, and the output $y, \hat{y}$ will look like the following for m anchor boxes but flattened into a single vector

$$\begin{bmatrix} p_c^1 & p_c^2 & \dots & p_c^m \\ p_x^1 & p_x^2 & \dots & p_x^m \\ p_y^1 & p_y^2 & \dots & p_y^m \\ p_h^1 & p_h^2 & \dots & p_h^m \\ p_w^1 & p_w^2 & \dots & p_w^m \\ c_1^1 & c_1^2 & \dots & c_1^m \\ \dots & \dots & \dots & \dots \\ c_n^1 & c_n^2 & \dots & c_n^m \end{bmatrix}$$

6.5.5 Region Proposal R-CNN, Fast R-CNN

instead of running the sliding window on everywhere in all cells, with the same slide, it can be more appropriate to narrow this down into very specific regions: first the input image is run through semantic segmentation function, dividing up the input into blobs(Regions proposal), then the Anchor boxes are run on top of them, but this process turns out to be very slow.

Fast R-CNN is a convolution implemtnation of it, and speeds up the process a bit.

6.6 Face Recognition

6.6.1 Recognition vs. Verification

in verification given an input image, name/ID, verify whether the input image is that of the claimed person, on the other hand, in a database of K persons, given an image, Recognize the image to one of the images in the database.

having 99% accuracy in the verification process is good enough threshold, meanwhile in recognition in database of 1000 persons, having 99% accuracy means that the classifier works with uncertainty of 1 to 10, there our classification model maps the image to 10 out of 1000 with equally likely probability, so accuracy in recognition need to approach 100 percent.

6.6.2 One-Shot learning

addressing the following criteria given a single input image for a person, as a single entry in the training-set, our model need to have accuracy approaching 100%. for new inputs to the trained model, how to scale?

also the shortcomming of the use of classification, is that it's trained on specific number of classes, or identities, but in the case of new input, the classification is expected to match the input to one of the softmax outputs, thus reducing the accuracy, to maintain the accuracy the model need to be trained for each new input, instead a siamese network is used to calculate the simiarity of new input against database records.

6.6.3 Similarity learning function

given two images I_1, I_2 , then similarity function $d(I_1, I_2)$ calculates the distance, and the distance inverse is how identical the two images are, such that $d = 0$ if $I_1 = I_2$, and verification is acceptable under

negligible distance threshold τ :

$$\begin{cases} same \leftarrow d(I_1, I_2) \leq \tau \\ different \leftarrow d(I_1, I_2) > \tau \end{cases}$$

a convolution example is the Siamese network:

6.6.4 Siamese Network

how to calculate the similarity distance?

.

for an convolution network ending with fully-connected with N nodes [FCN] layer, for each data-set entry $x^{(i)}$, we define $f(x^{(i)})$ to be the output of [FCN] on the entry $x^{(i)}$.

for two input images x^i, x^j from the data-set, the similarity function:

$$\|f(x^{(i)}) - f(x^{(j)})\|_2^2$$

.

so, how to define the loss function? using what is called a **Triplet Loss**

6.6.5 Triplet Loss

Loss function is defined on top of the similarity function d, in terms of distance between current image x^i , and the whole data-set images, the model can be trained on a single image called **Negative** x_i^n with large distance, and **Positive** x_i^p image with small distance, against the current image called **Anchor image** x_i^a , and the loss function for the triplet can be defined as $L(d(x_i^a, x_i^p), d(x_i^a, x_i^n))$.

but to prevent against the trivial solutions for triplets of the same persons, a marginal α is used, so the final loss equation is for N triplets:

$$\sum_i^N \|f(x_i^a) - f(x_i^p)\|_2^2 - \|f(x_i^a) - f(x_i^n)\|_2^2 + \alpha$$

this is exactly what makes the approach **one-shot**: enrolling a new person needs only a single reference image — we run it once through the trained network f and store its embedding, then recognition is a nearest-neighbour comparison of distances d . no retraining of the network is required when new identities are added.

6.6.6 Binary Classification

another variation is binary classification into “match”, or “not match”, can be performed into pair of images x^i, x^j , and label 0 (for different persons), or 1 (for the same person), with Sigmoid activation, defined as [FC] layer of size F:

$$\hat{y} = \sigma\left(\sum_k^F w_i \|f(x^i)_k - f(x^j)_k\| + b\right)$$

where w_i, b_i are training parameters.

or using the chi-square formula

$$\frac{(f(x^i)_k - f(x^j)_k)^2}{(f(x^i)_k + f(x^j)_k)}$$

the two branches share one set of weights — the same network f is applied to each input in parallel and only the embeddings are compared; this weight-tied “twin” design is what is called a **siamese network**, and it transfers directly from vision to NLP.

note that in NLP, the siamese network is used to match two different sentences, for example two search queries, with input being the embedding of the sentence, running through LSTM layer, and the distance is the cosine similarity function between the output, while the triplet function is defined with anchor vector of corresponding sentence, and positive, and negative vectors, such that $diff = d(A, P) - d(A, N)$, since the difference can be negative, cost function is required to be non-negative, a rectifier over the diff function is employed, and to capture the negative value difference, such that if the output for each negative difference is proportional to a positive value, and shift over the loss axis by adding small

constant α (similar to triplet loss in CNN), the final lost function is $relu(diff + \alpha)$, and the cost is the overall lost over all the training example $\sum_{i=1}^m L(A^{(i)}, P^{(i)}, N^{(i)})$.

training on hard triplet (are those satisfying $d(A, P) - d(A, N) = \epsilon$) are better for training to adjust the parameters to detect small differences, and the loss function is replaced by $L_{full} = L_1 + L_2$, $L_1 = relu(\text{mean Negative} - d(A, P) + \alpha)$, $L_2 = relu(\text{closest negative} - d(A, P) + \alpha)$, where mean Negative is the different with the negative values in the embedding matrix, and closest negative is the different with the closest negative in the embedding values.

6.7 Neural Style Transfer

first of all how to define style? style is the variations, or shades of specific content, and inside the neural network style transfer can be length of correlation between activations across different channels.

artistic style transfer, is the transfer of style of the Style image **S**, to the content image **C**, that outputs the generated image **G**.

cost function can be defined on the generated G, as follows:

$$J(G) = \alpha J_{content}(C, G) + \beta J_{style}(S, G)$$

using weighting parameters α, β to control how far the generated image is closer to either the content, or the style (note that a single weight is sufficient, for example α , and $(1 - \alpha)$) minimizing the cost between both the style, and the generated image, and the content, and the generated, thus merging the content, and the style images.

6.7.1 Generated image descent

gradient descent is minimizing $J(G)$, rather than weight parameters.

$$G = G - \frac{\partial J(G)}{\partial G}$$

the final generated image descent is as simple as:

1. initialize G randomly
2. use gradient descne to minimize $J(G)$

6.7.2 Content cost

using a middle layer l , not too shallow, not too deep,
 using pre-trained Convolution network, let $a_C^{[l]}, a_G^{[l]}$ be the activation of layer l on the content, and generated respectively, then we define

$$J_{content}(C, G) = \frac{1}{2} \|a_C^{[l]} - a_G^{[l]}\|^2$$

6.7.3 Style cost

6.7.4 Gram Matrix(style)

let $a_{i,j,k}^{[l]}$ be the activation at (i, j, k) , $G^{[l]}$ be the covariance (un-normalized) of different channels of size $n_c^{[l]}$, the style gram matrix can be formulated as:

$$G_{kk'}^{[l](S)} = \sum_{i=1}^{n_H^{[l]}} \sum_{j=1}^{n_W^{[l]}} a_{i,j,k}^{[l](S)} a_{i,j,k'}^{[l](S)}$$

generated gram matrix:

$$G_{kk'}^{[l](G)} = \sum_{i=1}^{n_H^{[l]}} \sum_{j=1}^{n_W^{[l]}} a_{i,j,k}^{[l](G)} a_{i,j,k'}^{[l](G)}$$

the final style cost function:

$$J_{style}^{[l]}(S, G) = \frac{1}{2(n_H^{[l]}n_W^{[l]}n_C^{[l]})^2} \|G^{[l](S)} - G^{[l](G)}\|_F^2$$

such that $\frac{1}{2(n_H^{[l]}n_W^{[l]}n_C^{[l]})^2}$ is a normalization term.

$$J_{style}(S, G) = \sum_l^L \lambda^{[l]} J_{style}^{[l]}(S, G)$$

with weight parameter λ .

Chapter 7

Recurrent Neural Networks RNN

7.1 Introduction

RNN has been making advance in the area of sequence modeling such as speech recognition, bio-informatics, music generation, natural language processing, and activity, and name recognition, given an input as a sequence, sequence models maps it to another sequence, for example translation of voices into words, or musical note, into a generated song, proteins in the DNA, etc.

7.2 Notation

in a sequence $x^{(i)}$, of the training data-set X, is dereferenced by $x^{(i)<t>}$, such that t is the t(th) (temporal element in the sequence) and represented by one-hot vector from a known Vocabulary V.

sequence $x^{(i)}$ have the corresponding length $T_x^{(i)}$.

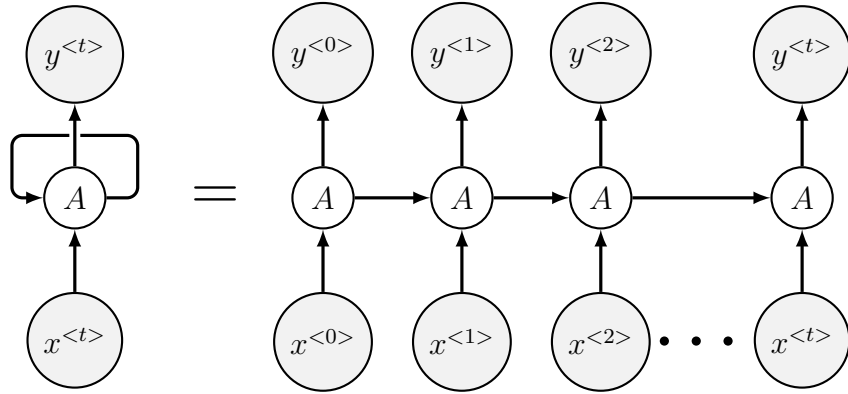
equivalently $y^{(i)<t>}$ is the element t of the $y^{(i)}$ output, with output sequence length $T_y^{(i)}$.

finally the weight parameter W_{ab} means wight for calculation of a after multiplication by b.

7.3 RNN

first of all, why NN isn't used in sequence modeling? NN is used in scenarios where there are shared features in the input, and where the input, and output are of the same length but in sequences it's not expected for the input features to be of similar features, and the length of input, and output different, therefore NN leads to less accurate model that is hard to train, instead in RNN, the layers are interconnected, and each input $X^{<i>}$ maps to $Y^{<i>}$.

7.3.1 RNN model



and the network is often initialized by random $A^{<0>}$ at the start of the network

this type of model are called unidirection RNN, compared to bidirectional RNN where the paths moves both ways.

the cost function is defined as the average of all output losses

$$J = \frac{1}{T} \sum_{i=1}^T \sum_{j=1}^{T_Y} y_j^{<t>} \log(y_j^{\hat{<t>}})$$

7.4 uni-directional RNN Forward propagation

$$a^{<t>} = g(W_{aa}a^{<t-1>} + W_{ax}X^{<t>} + b_a)$$

$$\hat{y}^{<t>} = g(W_{ya}a^{<t>} + b_y)$$

where g function is usually **tanh**, or **relu**.

$a^{<t>}$ is usually represented in vectorized notation into

$$a^{<t>} = g(W_a[A^{<t-1>}, X^{<t>}] + b_a)$$

where W_a is $\begin{bmatrix} W_{aa} & W_{ax} \end{bmatrix}$, and $[a^{<t-1>}, X^{<t>}] = \begin{bmatrix} a^{<t-1>} \\ X^{<t>} \end{bmatrix}$

$$\hat{y}^{<t>} = softmax(W_{ya}a^{<t>} + b_y)$$

$$L = \sum L^{<t>}(y^{\hat{<t>}}, y^{<t>})$$

notice that from the Bayesian chain-rule (see next chapter) that

$$P(y^{<t>} | y^{<1>} \dots y^{<t-1>}) = y^{\hat{<t>}}$$

7.5 uni-directional RNN backward propagation through time

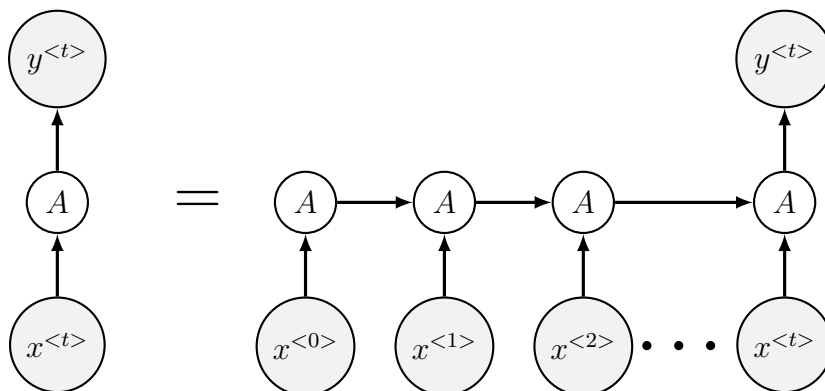
(derive)

7.6 Variations of RNN models

in the previous figure we discussed many-to-many uni-directional recurrent neural network architecture, a more general architecture is the bidirectional RNN where arrows goes both ways.

another variation is many-to-one, and it look like this

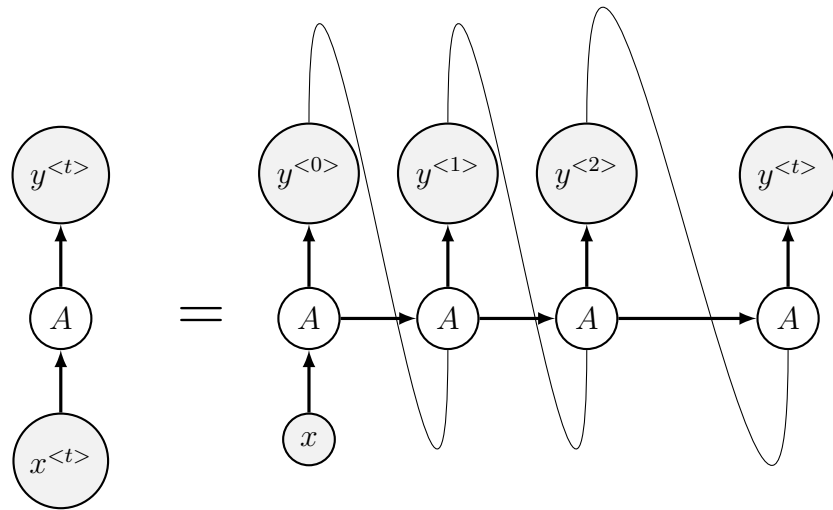
7.6.1 many-to-one RNN



notice that there is a single output the map the whole sequence to a single output, similar to the Neural network.

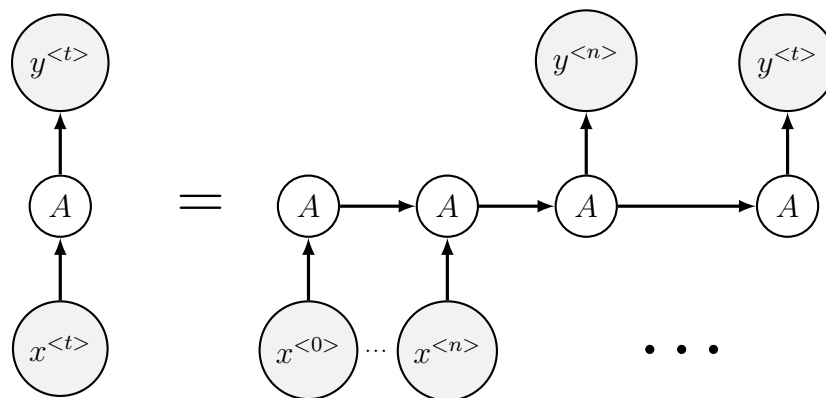
7.6.2 One-to-Many RNN

and finally one-to-many, input to next time step is sampled from the previous output:



for example in a music generation model, the input X could be the genre, or the initial note, or could be ϕ .

7.6.3 Many-to-Many of different input/output length



in the case of machine translation, the model is composed of an encoder consisting of the first input $X^{<0>}$ to $X^{<n>}$, and an output decoder consisting of the last output $y^{<n+1>}$ to $y^{<t>}$.

the plain encoder–decoder squeezes the entire input sequence into one fixed context vector, which degrades on long sentences; the **attention** mechanism (see the Attention Model section at the end of this chapter) lets each decoder step attend to a weighted combination of all encoder states instead of a single summary.

7.7 Language Model and sequence generation

In NLP sequence generation the many-to-many model of the same length is utilized, where each input (one-hot vector) representation of the word $X^{<t>}$ maps to $Y^{<t>}$: the probability of an input word given the prior words, can be the output of a **softmax** activation into the whole indices of the vocabulary.

after training the model on a large training example, the output sequence can be utilized to predict the most likely word after a given input sequence, for example for a new input sequence

$y^{<1>}, y^{<2>}, y^{<3>}$, what is $P(y^{<1>}, y^{<2>}, y^{<3>})$, using the chain-rule (see the appendix on probabilities) this term expands to $P(y^{<1>})P(y^{<2>}|y^{<1>})P(y^{<3>}|y^{<2>}, y^{<1>}) = y^{\hat{<1>}}y^{\hat{<2>}}y^{\hat{<3>}}$

7.8 Sampling novel sequences

it's possible to generate a novel sequence from the trained weights from a corpus, for example if the initial node $a^{<0>} = X^{<0>} = 0$, and the input $X^{<1>} \rightarrow X^{<T_x>}$ are taken from the previous output $y^{<0>} \rightarrow X^{<T_y-1>}$, for example $X^{<t>} = y^{<t-1>}$, then it output a sequence with writing style, and vocabulary similar to that of the corpus.

so if the the corpus vocabulary contain EOS_i or end of sentence token, sequence generator can stop after EOS_i , otherwise, it can set to generate a specific length of words (we will discuss pre-processing in NLP chapter).

in previous case the input is a vocabulary of words, it could be modeled on sequence of characters instead in similar way, in this case the vocabulary is the set of characters, and numbers, white space, and possibly punctuation.

7.9 Vanishing/Exploding gradients

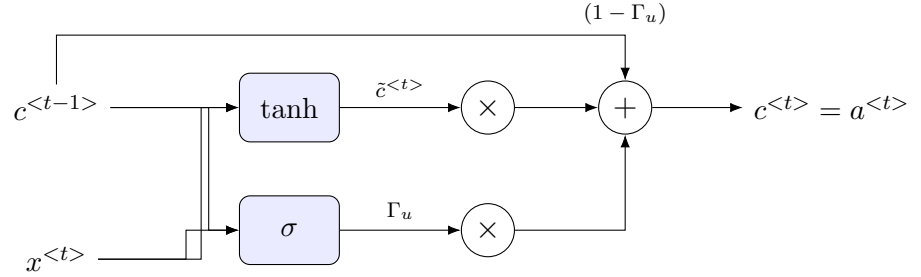
similar to the Neural network, RNN also can have Vanishing, and exploding gradients, it turns out that the latter is less common, in this case it leads to overflow of output, and usually solved by gradient clipping, or comparing the output against some threshold, then re-scale the gradient vector as to normalize it, but the former case of vanishing gradients is the most common, in a very long model.

7.9.1 Gated Recurrent Unit (GRU)

starting with an example from NLP, in English language, verbs are conjugated w.r.t the subject in the sentence, in a very

long text, keeping track of the subject is essential, but in the RNN which flows from the left to right can be substantially long, such forward, and backward propagation influences the advanced nodes is very negligible thus leading to vanishing gradients problems, to solve this, keeping a record of the such pivotal words is essential, for this GRU was invented.

the GRU cell takes the previous memory $c^{<t-1>}$ and the input $x^{<t>}$, computes a candidate memory $\tilde{c}^{<t>}$ and an update gate Γ_u , and blends the old and candidate memories to produce $c^{<t>}$:



7.9.2 GRU simplified

define memory cell C , $C^{<t-i>} = a^{<t-i>}$.

an estimate of the $C^{<t>}$ is notated with $\tilde{C}^{<t>}$.

the the estimated C is gated with Γ_u (update gate, sometimes denoted by Γ_i), multiplied together to generate the output $C^{<t>}$.

$$\tilde{C}^{<t>} = \tanh(W_c[c^{<t-1>}, X^{<t>}] + b_c)$$

$$\Gamma_u = \sigma(W_u[c^{<t-1>}, X^{<t>}] + b_u)$$

$$C^{<t>} = \Gamma_u * \tilde{C}^{<t>} + (1 - \Gamma_u) * C^{<t-1>}$$

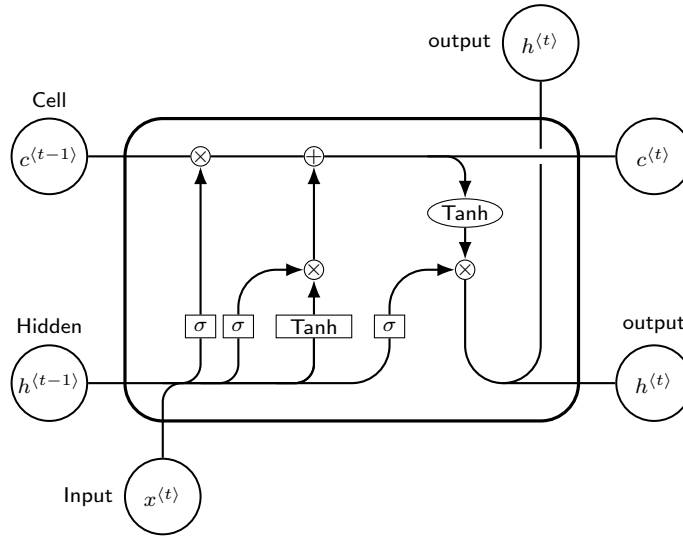
(note that $*$ denotes element-wise multiplication).

for a full GRU implementation relevance gate Γ_r is used to estimate the relevance of $C^{<t-1>}$ to calculate $C^{<t>}$

$$\Gamma_r = \sigma(W_r[c^{<t-1>}, X^{<t>}] + b_u)$$

$$C^{\tilde{<t>}} = \tanh(W_c[\Gamma_r * c^{<t-1>}, X^{<t>}] + b_c)$$

7.9.3 Long Short Term Memory (LSTM)



$$C^{\tilde{<t>}} = \tanh(W_c[c^{<t-1>}, X^{<t>}] + b_c)$$

$$\Gamma_u = \sigma(W_u[a^{<t-1>}, X^{<t>}] + b_u)$$

$$\Gamma_f = \sigma(W_f[a^{<t-1>}, X^{<t>}] + b_f)$$

is the forget gate used instead of $(1 - \Gamma_u)$.

$$\Gamma_o = \sigma(W_o[a^{<t-1>}, X^{<t>}] + b_o)$$

is the output gate

$$C^{<t>} = \Gamma_u * C^{\tilde{<t>}} + \Gamma_f * C^{<t-1>}$$

$$a^{<t>} = \Gamma_o * \tanh(C^{<t>})$$

one notice here is that in Γ_o instead of $X^{<t>}$ you could use $C^{<t-1>}$ instead, and known as **peephole connection**

7.9.4 Back-propagation

back-propagation through an LSTM cell receives the incoming gradients $da^{<t>}$ (from the output/next layer) and $dc^{<t>}$ (from the next time-step), and produces the gate gradients together with $da^{<t-1>}$, $dc^{<t-1>}$ passed to the previous step.

the output gate first routes $da^{<t>}$ into the cell state, so the cell gradient accumulates

$$dc^{<t>} += da^{<t>} * \Gamma_o * (1 - \tanh^2(c^{<t>})).$$

the pre-activation gradients of the four gates (each folding in the derivative of its own non-linearity) are

$$d\tilde{c}^{<t>} = dc^{<t>} * \Gamma_u * (1 - \tilde{c}^{<t>2}),$$

$$d\Gamma_u = dc^{<t>} * \tilde{c}^{<t>} * \Gamma_u(1 - \Gamma_u),$$

$$d\Gamma_f = dc^{<t>} * c^{<t-1>} * \Gamma_f(1 - \Gamma_f),$$

$$d\Gamma_o = da^{<t>} * \tanh(c^{<t>}) * \Gamma_o(1 - \Gamma_o).$$

stacking the concatenated input $[a^{<t-1>}, x^{<t>}]$, each weight matrix gradient is the outer product of the corresponding gate gradient with that input, e.g.

$$dW_c = d\tilde{c}^{<t>}[a^{<t-1>}, x^{<t>}]^T, \quad db_c = \sum d\tilde{c}^{<t>},$$

and similarly for W_u, W_f, W_o .

finally the gradients handed to the previous step are

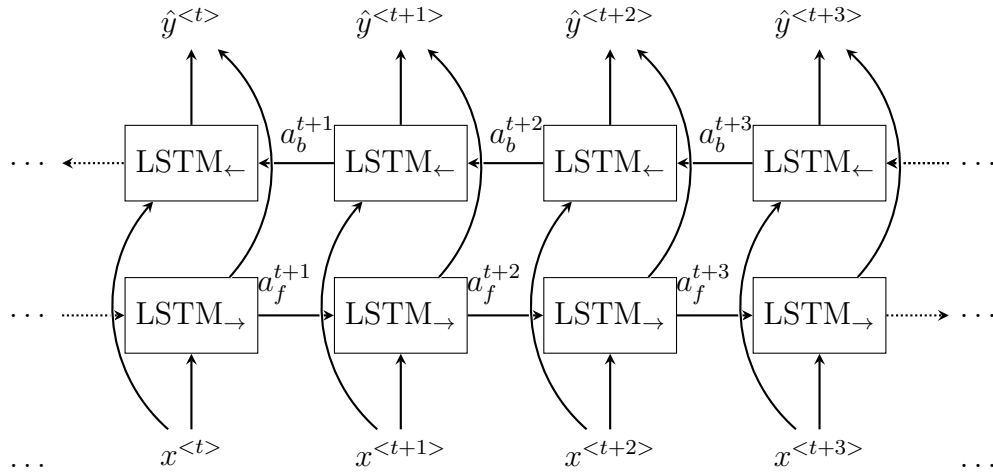
$$dc^{<t-1>} = dc^{<t>} * \Gamma_f, \quad da^{<t-1>} = W_c^T d\tilde{c}^{<t>} + W_u^T d\Gamma_u + W_f^T d\Gamma_f + W_o^T d\Gamma_o$$

(keeping only the rows of each W_* that multiply the $a^{<t-1>}$ block of the concatenation).

7.9.5 Bidirectional RNN

bidirectional RNN runs forward propagation both forward, and backward, utilizing two activation unites $a_f^{<t>}$, and $a_b^{<t>}$, instead of just $a^{<t>}$, where the propagation flows from $a_f^{<i>} \rightarrow a_f^{<i+1>}$, at the same time, from $a_b^{<i+1>} \leftarrow a_b^{<i>}$, to estimate

$$y^{<t>} = g(W_y \begin{bmatrix} a_f^{<t>} \\ a_b^{<t>} \end{bmatrix} + b_y)$$



this model require first processing all the sequence input, before generating any output, with advantage of both backward, and forward contextual semantics awareness, and disadvantage that output is generated after processing all the input X.

7.10 Deep RNNs

a deep RNN stacks several recurrent layers: the activation of layer l at time t , written $a^{[l]<t>}$, is fed both forward in time (to $a^{[l]<t+1>}$) and upward in depth (to $a^{[l+1]<t>}$). because each layer is already unrolled over the whole sequence, RNNs are rarely stacked more than 3 deep.

forward propagation: for each layer $l \in [1, L]$ and step t ,

$$a^{[l]<t>} = g(W_a^{[l]}[a^{[l]<t-1>}, a^{[l-1]<t>}] + b_a^{[l]}),$$

where $a^{[0]<t>} = x^{<t>}$ is the input and $a^{[l]<0>}$ is the (usually zero) initial state of layer l ; the top-layer output feeds the usual per-step prediction $\hat{y}^{<t>} = \text{softmax}(W_y a^{[L]<t>} + b_y)$.

backward propagation is back-propagation through time applied to the stack: at each (l, t) the gradient $da^{[l]<t>}$ collects two contributions — one from the same layer at the next time-step and one from the layer above at the same step — and is then pushed through g' to accumulate $dW_a^{[l]}, db_a^{[l]}$, exactly as in the single-layer case.

7.11 Word Representation

previously we discussed one-hot vector representation of a word in NLP models(we will elaborate more on that in following chapters), as a representation of a word in vector of size V , which is the length of the vocabulary, each word is denoted by $O_{wordindex}$, but this sparse representation have drawbacks, that the dot product(degree of alignment) isn't captured, so any two words, no matter how similar will have zero dot product, for example the pair Man/Woman, King/Queen, Apple/Orange aren't distinguished from each others, instead a high dimensional featured representation is used coined **Word Embedding**.

7.11.1 Word Embedding

word embedding for dictionary V is a matrix where each word is represented by a column vector matched against features at each row. in this case, we can use rules of Linear Algebra applied on word embedding representation, as for example to calculate the distance between two different words, so in the pair Queen/King, or Woman/man we expect the distance between the words in each pair to be the same with negligible difference, and this distance is called **Cosine Similarity** (we will elaborate on that in following chapters).

7.11.2 Named Entity recognition

in the following two sentences

1. Sally Johnson is an orange farmer.
2. Robert Lin is an apple farmer.

only with word Embedding a trained BRNN model on a large text corpus, where it encounter the first sentence, can deduce from the second example the “Robert Lin” is a name of person.

to use such model, it first trained on large data-set, then with transfer learning, new tasks are added with moderate size data-set for example 100k words, as analogous to convolution NN in computer vision, where a trained model with large data-set can be utilized with image encoding to generalize to new task.

for a Vocabulary of V words, the **Matrix Embedding** E of N -dimensional features, is consistent of V word embedding, for example for word w^i , if it's embedding it's e_{w^i} , then $EO_{w^i} = e_{w^i}$ of size N .

7.11.3 Learning word embedding

How word embedding is generated for a new word? given Matrix Embedding? the standard trick is to set up a fake prediction task (predict a word from its context, or a context word from a target) and read the learned weight matrix E as the embedding — the embedding is a by-product of training that language-model-like network.

In a sequence of words $w_1, w_2, \dots, w_{n_1}$, prediction of the **nth** word w_n can be the output of a Neural Network, feed by window **W** of preceding word embedding, given a Matrix Embedding E of N features, and sentence, with corresponding one-hot vector representation of **V** words vocabulary, each vector word embedding e_{w^i} is calculated, and feed as input to the neural network. the final neural network would be of input size $W \times N$, feed to neural network of L layers (usually 1 layer), with final SOFTMAX output of length V , and produces the predicted one-hot vector representation of the **nth** word w_n .

7.11.4 Word2Vec

7.11.5 Skip gram

skip-gram word embedding generation fixated a target word w_t , or for simplicity, rather than feeding the preceding sequence in window W , it randomly samples a context word w_c , or c for simplicity, from a window of length W , with equal length input/output layers, with middle SOFTMAX activation layer.

7.11.6 hierarchical SOFTMAX

for a small vocabulary V of even 10k words, generating a word embedding for each words $P(w_t|w_c)$ can be quite expensive summing over the entire vocabulary for each target word:

$$p(w_t|w_c) = \frac{e^{\theta_t^T e_c}}{\sum_{j=1}^V e^{\theta_j^T e_c}}$$

notice here the bias term is ignored, where θ_k is the weight parameter associated with word w_k for $i \in [1-V]$. so instead the hierarchical SOFTMAX sorts the context words in tree, and uses a binary-search-like algorithms to narrow down the search to closest word region.

concretely, the words are arranged as the leaves of a binary tree and each internal node holds its own logistic classifier; the probability of a word is the product of the left/right decisions along the root-to-leaf path, so evaluating (and normalizing) one word costs $O(\log_2 V)$ classifiers instead of the full $O(V)$ sum. a Huffman tree (frequent words nearer the root) makes the average path even shorter.

secondly in the random sampling, the heuristic is not to choice uniformly, ending up repeating the calculation for the most frequent words for example the STOP words “the”, or “of” as context words.

7.11.7 Negative Sampling

another variant, yet faster than SOFTMAX embedding generation network is Negative Sampling, it solves the expensive activation, so instead of summing over the whole Vocabulary, the window of K words from a data-set of input X context/target words map, and output binary vector Y,

$$\begin{cases} 1 \leftarrow \text{context and target matches} \\ 0 \leftarrow \text{otherwise} \end{cases}$$

the sampling of the k words is chosen from the distribution:

$$P(w_i) = \frac{f(w_i)^{\frac{3}{4}}}{\sum_{j=1}^V f(w_j)^{\frac{3}{4}}}$$

for f being the frequency function, (this proven to be efficient).

k is changed depending on the size of the data-set for a small data-set it usually 5-20, for large-dataset, it's less then 5, for example the input vector for the target word w_t from randomly chosen context words with X-Y being for example:

$$\begin{bmatrix} w_c^1 \\ w_c^2 \\ \vdots \\ w_c^k \end{bmatrix} \rightarrow \begin{bmatrix} 1 \\ 0 \\ \vdots \\ 0 \end{bmatrix}$$

for a given data-set, it's possible to randomly choose words from the dictionary, and assign it as negative (with value zero), and added it to the X-Y mapping,

then similar to the skip-gram algorithm, the embedding of each word vector is feed to the neural network, but instead of softmax activation, logistic activation used instead:

$$P(y = 1|c, t) = \sigma(\theta_t^T e_c)$$

7.11.8 GloVe (global vector for word representation)

even simpler word representation is the GloVe algorithm, a descent of the neural word embedding, and skip-gram.

for a specific context function, input X , is the frequency of the one target word appearing in the context of another, such that X_{ij} is the frequency of the w_j appearing in the context of w_i .

Glove cost function is thus defined as follows

$$J(\theta, e) = \sum_{i=1}^V \sum_{j=1}^V f(X_{ij})(\theta_i^T e_j + b_i + b_j - \log(X_{ij}))^2$$

in gradient descent both the weight θ , and the embedding e are updated, moreover after each word, the embedding is normalized as follows $e_w = \frac{e_w + \theta_w}{2}$

it's worth noting that for invertible matrix A , a transformation over the vector space of the embedding $(A\theta_i)^T(A^{-T}e_j) = \theta_i^T e_j$, meaning different transformation over the embedding vector can lead to the same evaluation of the cost, and therefore with different interpretations on the single vector space, that doesn't guarantee that specific embedding will be human-interpretable.

7.12 Example: Sentiment Classification

for a sequence of N words review, for example a review of a product, ignoring the sequence order, the problem can be modeled in a simple SOFTMAX activation layer of the average, or sum of the embedding of the sequence words $z = \sum_{i=1}^N e_i$, while Y is a 5 star classification vector of length 5, $Z = W^T \text{average}(X) + b$, $\hat{y} = \text{softmax}(Z)$, $L = Y \cdot \log(\hat{Y})$ the problem with this model, it fails to accurately classify long sentences, sentences that aren't straight-forward, to maintain the sequence, many-to-one RNN is used instead, and feed with the word embedding as the input, with output softmax layer.

7.13 Sequence To Sequence Model

sequence to sequence model is a generative model, generating an output sequence $y^{<1>}-y^{<T_y>}$, or

$$P(y^{<1>}, \dots, y^{<T_y>} | x^{<1>}, \dots, x^{<T_x>})$$

from input sequence $x^{<1>}-x^{<T_x>}$, an interesting example is image captioning, taking Alexnet network SOFTMAX output, can fit as a an input sequence, to be trained on captioning data-set. for example T_x can be softmax of length 4096, and T_x is the captioning length can be set to 10 words.

a single input sequence $\mathbf{X}$ can map to different possible output sequence $\mathbf{Y}$, sequence to sequence model aim for the maximum probability

$$\operatorname{argmax}_{y^{<1>}, \dots, y^{<T_y>}} \prod_{t=1}^{T_y} P(y^{<t>} | X, y^{<1>}, \dots, y^{<t-1>})$$

to explain the complexity analysis, let's pick an example, for example in NLP machine translation, the **exact** argmax (scoring every possible output sequence) has complexity $O(V^{T_y})$, where V is the vocabulary size, for example in a output as small as 10 sequence words, from a moderate vocabulary of length 100k, then the sample space is 100000^{10} ! which is infeasible for a moderate application, instead a non-optimal heuristic search algorithm is usually employed for example **beam search**.

to settle the cost question directly: **greedy** decoding (keep the single best word at each step) costs $O(V \cdot T_y)$; **beam search** with width B keeps the B best partial sequences and at each step scores $B \cdot V$ extensions, so it costs $O(B \cdot V \cdot T_y)$ — exactly B times greedy, and greedy is the special case $B = 1$. so beam search does *not* reduce cost relative to greedy; it spends a constant factor B more in exchange for a higher-probability sequence, while still being astronomically cheaper than the exact $O(V^{T_y})$ search.

7.13.1 Beam search

noticing in the previous example the in almost all cases, in machine translation example from a vocabulary of size V , only a small fraction of words can have reasonable probability, so it's possible to set a Window/Width W for example of $W = 5$ words to search instead of the whole vocabulary at the initial step, so to instantiate the network N times, so at first sequence $Y^{<1>}$ there are only W possibilities are fit $P(y^{<1>}|X)$, similarly at second step $Y^{<2>}$ there are equivalently maximum W words $P(y^{<1>}, y^{<2>}|X)$, at the second step another instantiation of the network, leading to step 3, for sequence $Y^{<3>}$, $P(y^{<1>}, y^{<2>}, y^{<3>}|X)$, and so on, this shrink down the complexity to $O(W \times V \times T_y)$.

7.13.2 normalization

in a long sequence model, the argmax output for a small values of X, Y can quickly diminishes to zero, to prevent this, the mean the log of the likelihood is implemented instead:

$$\operatorname{argmax}_{y^{<1>}, \dots, y^{<T_Y>}} \frac{1}{T_y^\alpha} \sum_{t=1}^{T_y} \log P(y^{<t>}|X, y^{<1>}, \dots, y^{<t-1>})$$

(note that α is tuning parameter).

beam search is non-optimal compared to DFS, or BFS algorithms, but with advantage of being faster, and the reason for that, is it never search the whole sequence space.

how to choose the beam Width? beam width is a trade-off between speed, and accuracy, in practice it's usually in the range 5-10.

7.13.3 Error Analysis

in many-to-many RNN network employing Beam search, it could be that the window/width of beams search at fault, or the RNN at fault, to decide it wise to keep a boolean

flag for the following to look at during development, given an input sequence X , and ground truth table output Y , the beam search is at fault, if for the output $\hat{Y}$, $P(Y|X) > P(\hat{Y}|X)$, otherwise, then the RNN is at fault.

7.13.4 Bleu Score

the previous example still proceeds to explain Bleu Score metric which is essential for such example where the output sequence can take different valid output, for example for input sentence, from English language, the Spanish translation, can varies, to evaluate the machine translation MT, against human translation references, through combined bleu score:

$$Bp \exp\left(\frac{1}{4} \sum_{n=1}^4 P_n\right)$$

where **Bp** is coined brevity penalty, and defined as

$$\begin{cases} 1 & \text{if length of MT} > \text{reference length} \\ \exp\left(\frac{1 - \text{reference length}}{\text{MT length}}\right) & \text{otherwise} \end{cases}$$

bleu score p_n for set for references output, and MT output, defined using two function, count for counting the n-grams in the MT combined, and references, and $count_{clip}$ for counting the number of n-grams in any of the references, and thus defined as:

$$p_n = \frac{\sum_{\text{n-gram} \in \hat{y}} Count_{clip}(\text{n-gram})}{\sum_{\text{n-gram} \in \hat{y}} Count(\text{n-gram})}$$

7.14 Attention Model

first let the bi-directional RNN output for time step t be $a^{<t>} = (a_f^{<t>}, a_b^{<t>})$

attention model takes the output sequence of a bi-directional RNN $a^{<1>}, a^{<2>}, \dots, a^{<T_x>}$ as the input sequence of a second layer LSTM-RNN (called hidden

state S), where each output is associated with attention/weights given to K input sequences $a^{<t>}, a^{<t+1>}, \dots, a^{<t+k>}$, and output sequences,

$\begin{bmatrix} \alpha^{<t,t'>} \\ \alpha^{<t,t'+1>} \\ \vdots \\ \alpha^{<t,t'+k>} \end{bmatrix}$ the first layer is initialized with $a^{<0>}$, and second hidden

state layer is initialized with $s^{<0>}$ (note the second index is for input sequence, and first index is the index of the second LSTM layer index).

7.14.1 Attention

the attention commit to the following constraints, $\sum_t \alpha^{<t,t'>} = 1$ for all t' time steps. and the input to the second RNN layer $c^{<t>} = \sum_t \alpha^{<t,t'>} a^{<t>}$ for all t' .

since the attention is dependent on the current state at time t' for the output of first LSTM layer, and the previous state of the hidden layer, and since this parameter isn't known, a softmax activation is used to train, and estimate this parameter $e^{<t,t'>}$, in order to calculate

$$\alpha^{<t,t'>} = \frac{\exp(e^{<t,t'>})}{\sum_{t'=1}^{T_x} \exp(e^{<t,t'>})}$$

and the parameter $e^{<t,t'>}(a^{<t'>}, s^{<t-1>})$ is estimated for each time step.

Chapter 8

Probabilistic Graphical Models (PGM)

8.1 Introduction

first, what Models are? models are declarative representation of the world, **probabilistic models:** are representation of the uncertainty of the model. PGM is graphical representation of a phenomena, that can be used with other input data with other algorithms to take decisions, in classification, segmentation, or recognition. and the reasons for uncertainty :

1. partial knowledge of state of the world
2. noisy observations
3. phenomena not covered by the model
4. inherent stochasticity: that the world is inherently stochastic, and therefor the model.

therefore the model is probabilistic and feed with random input variables mimicking uncertainty with their joint distribution.

examples on graph are Bayesian network, represented with directed graph, and Markov networks (un-directed graph, usually weighted).

8.1.1 preliminaries

given random variables (for example I,D,G, being the data sampled from a class I(intelligence), D(difficulty), G(grade) the joint distribution $P(I,D,G)$, and represented as a vector of joint distribution.

reduced random variable: is the joint distribution under certain conditions for subset of the random variables for example $p(I, D, G^1)$ of the binary random variable G(taking the values 0,1), G^1 is the random variable $G = 1$. and $p(I, D, G^1)$ is the reduced distribution to G^1 values.

re-normalized RV: is the process of normalizing the distribution after reduction.

marginalization of RV: is the distribution of one RV summed over the distribution of another for example $P(D) = \sum_{i=1}^n P(D|I)$.

8.1.2 factors

consider the vector variables A,B,C with arbitrary values called scope, factor are the building block, or the input for our model for probabilistic representation in high-dimensional spaces.

factor is a map from set of random variable **scope**(parameters) to real value $\phi(X^0, X^1, \dots, X^n) \rightarrow \mathbb{R}$ for example the joint distribution $P(I, D, g^1)$ is a factor, and the scope in the case is I,D since g^1 is set to g^1 binary random variable constant value, factors aren't necessarily of probability scope, but could be for any values.

factor product: $\phi_1(A, B)\phi_2(B, C) = \phi_3(A, B, C)$ multiplication done on the real scalar values.

factor marginalization: from distribution $\phi(A, B, C)$, to marginalize over B, $\phi(A, C) = \sum_{i=1}^{n=\#B} \phi(A, C)$ ($\#B$ means size of variable B).

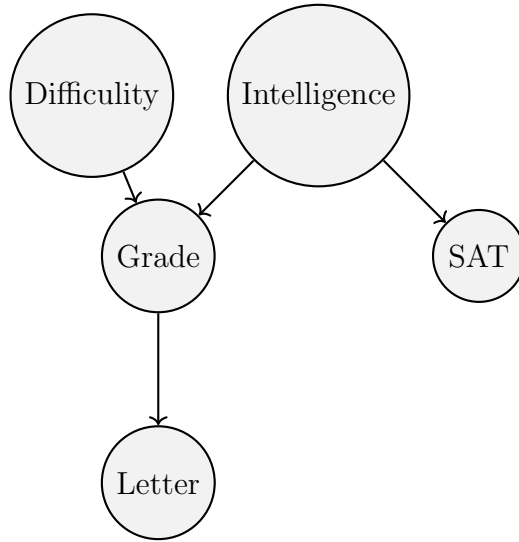
reduced factor: reduced factor over C^1 is $\phi(A, B, C^1) = \phi(A, B, C) \leftarrow C = 1$.

Conditional Probability Distribution (CPD): is the conditional probability with a joint probability distribution, for example $P(G|I, D)$.

8.2 Bayesian Network Fundamentals

for student grading with vector random variables, G (grades), D (course difficulty), I (intelligence), S (sat score), L (reference recommendation letter).

8.2.1 constructing dependencies



first graph is constructed to mimic the phenomenon, which is the student score: in which the grade is dependent on both D, I , $p(G|D, I)$, and S is dependent on I , $P(S|I)$, recommendation letter is dependent on the grade $P(L|G)$. so in this constructed graph, factorization of the joint distribution $P(D, I, G, S, L) = P(D)P(I|D)P(G|D, I)P(S|D, I, G)P(L|D, I, G, S)$ from the chain rule (see appendix on probabilities). from the graph $P(I|D) = P(I)$, $P(S|D, I, G) = P(S, I)$, $P(L|D, I, G, S) = P(L|G)$, therefor $P(D, I, G, S, L) = P(D)P(I)P(G|D, I)P(S|I)P(L|G)$.

here example for the graph truth table for the probability dis-

tributions for example $P(D) \begin{array}{|c|c|} \hline d^0 & d^1 \\ \hline 0.6 & 0.4 \\ \hline \end{array}$, $P(I) \begin{array}{|c|c|} \hline i^0 & i^1 \\ \hline 0.7 & 0.3 \\ \hline \end{array}$, $P(S)$

	s^0	s^1			g^1	g^2	g^3			l^0	l^1
i^0	0.95	0.05	, P(G)	i^0, d^0	0.3	0.4	0.3	, P(L)	g^1	0.1	0.9
i^1	0.2	0.8		i^0, d^1	0.05	0.25	0.7		g^2	0.4	0.6
				i^1, d^0	0.9	0.08	0.02		g^3	0.99	0.01
				i^1, d^1	0.5	0.3	0.2				

now what is $P(d^0, i^1, g^3, s^1, l^1)$? from the last derived equation it's $P(d^0)P(i^1)P(g^3|d^0, i^1)P(s^1|i^1)P(l^1|g^3)$ from the tables it's $0.6*0.3*0.02*0.01*0.8$.

A Bayesian network is a directed a-cyclic graph (DAG) such that each node of the graph is a representation of the underlying random variable. such that for each node X_i (random variable) a CPD is $P(X_i|Par_G(X_i))$ where Par_G is a function mapping node (i) random variable to it's parent nodes in the directed graph G

the probability distribution at any node with X_i representation is $P(X_1, ..., X_n) = \prod_i P(X_i|Par_G(X_i))$ (chain rule see the appendix).

BN distribution build off probability distribution P is expected to be $P > 0$, and $\sum P = 1$. for example in the prior example $\sum_{D,I,G,S,L} P(D, I, G, S, L) = 1$.

Probability P factorizes over G $\iff$ it's represented by chain rule.

8.3 Reasoning Patterns on BN

Causal reasoning: is reasoning from bottom up, for example $P(l^1) \approx 0.5$, $P(l^1|i^0) \approx 0.39$, $P(l^1|i^0, d^0) \approx 0.51$.

Evidential Reasoning: is top down reasoning toward the parent nodes, in a class where student gets C with initial $P(i^1) = 0.3$, $P(d^1) = 0.4$, then the hypothesis that the student is smart given the grades $P(i^1|g^3) \approx 0.08$ goes down, and the hypothesis that the class is difficult given the grade $P(d^1|g^3) \approx 0.63$ goes up.

Intercausal Reasoning: in relational evidential reasoning, for example what is the probability of the intelligence of a student given the grade, and that it was difficult $P(i^1|g^3, d^1) \approx 0.11$ it went up from 0.08.

8.4 Flow of Probabilistic Influence

it was discussed in previous section that both causal reasoning, and evidential reasoning between the parent, and child node influence each others directly, for example for X, Y variables $X \rightarrow Y$ e causally affect Y , and $X \leftarrow Y$ evidentially affect Y . similarly in presence of via node in the case of $X \rightarrow W \rightarrow Y$, then X influences Y via W , similarly $X \leftarrow W \leftarrow Y$ then X influences Y .

in the case in direct edges via nodes even from the same graphical level, for example $X \leftarrow W \rightarrow Y$ this can be reduced to $X \leftarrow W$ then X influences W , and $W \rightarrow Y$ then W influences Y , therefore X influences Y , in the student example the observation of the sat score influences the grade.

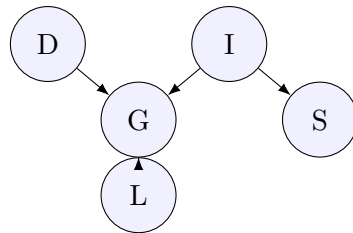
meanwhile in the case of **V-structure** $X \rightarrow W \leftarrow Y$ in this case X influences W , and Y influences W , but X , doesn't influence Y .

8.5 d-separation

X , and Y are d-separated in the graph G , if there is no flow of influence between them, and it's denoted by $d\text{-sep}_G(X, Y|Z)$, interpreted as X , and Y are separated on the graph G , or there is no flow of influence between them.

to look for active trails, we search the graph from V-structures.

the running example is the “student” network over Difficulty (D), Intelligence (I), Grade (G), SAT (S) and Letter (L):



here $D \rightarrow G \leftarrow I$ is a V-structure: D and I are marginally independent, but observing G (or its descendant L) opens

the trail and makes them dependent; conversely D and S are d-separated given I.

If P factorizes over G, and $d - sep_G(X, Y|Z)$ then P satisfies $(X \perp Y|Z)$. if there is d-separation between X, and Y given Z, then there is no flow of influence given Z, and from definition of independence $P(X \perp Y|Z) = P(X|Z)P(Y|Z)$.

chain rule expansion of $P(D, I, G, S, L) = P(D)P(I|D)P(G|I, D)P(S|G, I, D)$ from flow of influences in G it reduces to

$$P(D)P(I)P(G|D, I)P(S|I)P(L|G)$$

$$\begin{aligned} P(D, S) &= \sum_{G, L, I} P(D)P(I)P(G|D, I)P(S|I)P(L|G) \\ &= \sum_I P(D)P(I)P(S|I) \sum_G (P(G|D, I) \sum_L (L|G)) \\ &= P(D) \left(\sum_I P(I)P(S|I) \right) \\ &= P(D)P(S) = \phi_1(D)\phi_2(S) \end{aligned}$$

then $P \models D \perp S$

any node is d-separated from it's non-descendants given it's parents, that is the same rule used to simplify the chain rule joint distribution of the graph G.

8.5.1 I-map

d-separation in G $\rightarrow$ satisfies corresponding independence:

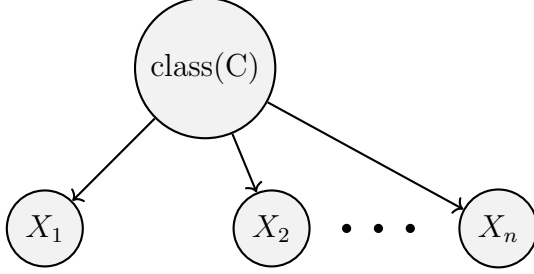
$$I(G) = (X \perp Y|Z) : d - sep_G(X, Y|Z), X, Y, Z \in G$$

if P satisfies I(G) then G is an I-map (independency map) of P.

if G is an I-map for P, then P factorizes over G.

8.6 Naive Bayes Model

naive base is a graph of independent Variables mapping to the same parent.



$$(X_i \perp X_j | C) \forall X_i, X_j$$

$$P(C, X_1, \dots, X_n) = P(C) \prod_{i=1}^n P(X_i | C)$$

given two observations of the input features X , we can define the ratio probability of outcome of two observation:

$$\frac{P(C = c^1 | x_1, \dots, x_n)}{P(C = c^2 | x_1, \dots, x_n)} = \frac{P(C = c^1)}{P(C = c^2)} \prod_{i=1}^n \frac{P(x_i | C = c^1)}{P(x_i | C = c^2)}$$

(note the first term denoted by the prior, and the second is the odd ratio).

8.7 Template Model

in real world phenomenons, it's always the case that Bayesian representation of a single isolated example is part of larger interconnected network, with shared features, or parameters, for example in NLP sequence model, the previous word, or character influences the current word/character, even the following sequence can resolve the function of current verb, or identity of the none (in named entity recognition for example), similarly in genetic inheritance model, the phenotypes, and genotypes are shared between different branches of wider family, in

robot localization, the current position is influenced by control input, prior speed, and position, in image recognition, it's redundant to create a model for each pixel, but rather through is build off the principle of super pixels, where a blob of pixels with shared features are mapped together into a single network.

A template variable $X(u_1, \dots, u_k)$ is instantiated multiple times, for example the location in robot localization at specific time step, person's genotypes, pixel in a blob, or segment, etc.

8.7.1 Distribution over Trajectories

the Template Model is based of time sequence, or distribution over the trajectory as function of time, with time granularity Δ , such that $X^{(t)}$ is the value of X at time Δt . $X^{(t:t')} = X^{(t)}, \dots, X^{(t')}(t \leq t')$

8.7.2 Markov chain

in a network with trajectory of time Δt at state $X^{(0:T)}$ with probability:

$$P(X^{(0:T)}) = P(X^{(0)}) \prod_{t=0}^{T-1} P(X^{(t+1)} | X^{(0:t)})$$

the markov assumption is that the state at time $t + 1$ is independent of previous states in the interval $[0-(t-1)]$ given the state at time t , or

$$(X^{(t+1)} \perp X^{(0:t-1)} | X^{(t)})$$

.

this assumption can be generalized in the following statement **“once you Know your present, the past doesn’t Matter!”**

thus we can simplify the first equation into:

$$P(X^{(0:T)}) = P(X^{(0)}) \prod_{t=0}^{T-1} P(X^{(t+1)} | X^{(t)})$$

but you should accept this assumption with a grain of salt, so if the current state doesn't capture the past in full, in this case, this assumption breaks, for example future state of robot position is dependent not just of the position, but velocity as well.

so only the likelihood $P(X^{(t+1)}|X^{(t)}) = P(X'|X)$ and the prior $P(X^{(0)})$ on the interval $[0-T]$ determine the following state, being **Invariant to the time**.

the edges between different time steps are coined:

1. **Intra-time slice:** the edges the the same time t
2. **inter-time slice:** the edges between time t , and time $t+1$.
3. **persistence-time slice:** is inter-time slice but between the same state variables.

8.7.3 Dynamic Bayesian Network DBN

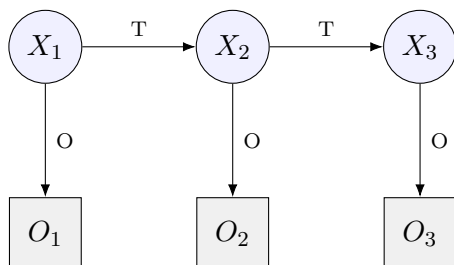
Dynamics network is a generalization of a template model Bayesian network defined in trajectory of time steps of arbitrary range, where next state model defined in terms of current states, and priors.

8.7.4 Hidden Markov Model (HMM)

HMMs are subclass of **DBNS** with hidden edges either from each state to itself, or from the state at time t_i to arbitrary time t_j .

HMM start from state X to X' through a transition state, and outputs an observation at X' , through the HMM network, transition, and observation are wrapped in two matrices Transition, and Observation matrices. (we will elaborate more on that in following chapters)

unrolled over time the hidden states X_t form a Markov chain (horizontal transition edges) while each emits its observation O_t (vertical emission edges):



8.8 Structured CPD

previously used tabular representation of the CPD isn't suitable for large network, with node parented by large number of parent nodes, any representation satisfies the general definition would suffice.

a general CPD $P(X|Y_1, \dots, Y_n)$ specifies distribution over X , for each assignment $Y_1, \dots, Y_n$ can use any function to specify a factor $\phi(X, Y_1, \dots, Y_n)$ such that $\sum_x \phi(X, Y_1, \dots, Y_n) = 1$ for all $Y_i, \dots, Y_n$.

under the general form, usually there are different models used for example:

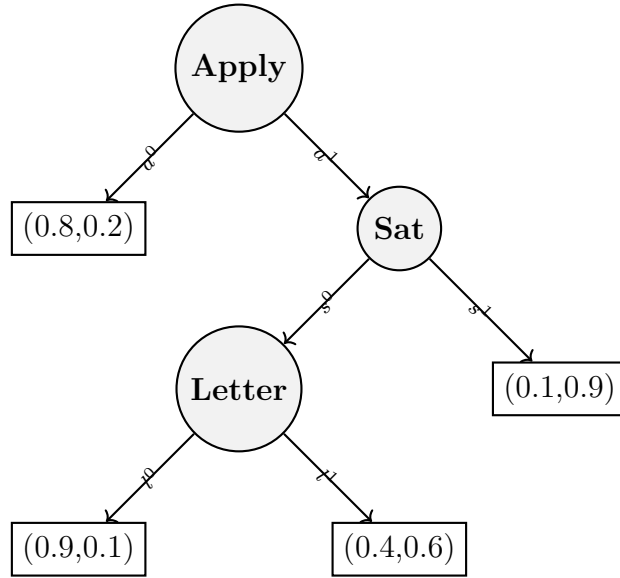
1. Deterministic: is set of $Y_1, \dots, Y_n$ random variables.
2. Tree-structured: is conditional CPD.
3. Logistic: logistic activation of the variables.
4. Noise OR/AND: noise CPD variables.
5. Linear Gaussian: continuous variable CPD.

context-specific independence: for two independent variables X, Y given Z , are called context independent given context c , where the context is some condition, defined by a function, then CPD satisfies:

$$(X \perp_c Y | Z, c)$$

. examples on this will be in the Tree CPD.

8.8.1 Tree CPD



in the previous student example, let's add two nodes, Apply (probability of the student applying for a job), and Job(probability of getting a job), in presence of Sat node, and Letter node.

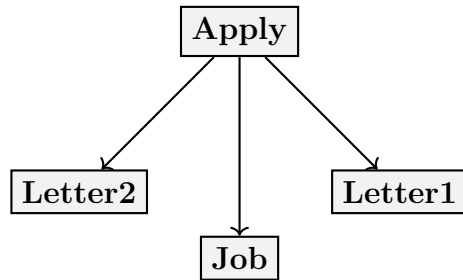
Tree CPD is CPD with dependencies, and possibly context-specific in-dependencies, on each node's parents, thus flow passes from the parent to child node.

this graph is called **context-specific CPD Tree** representation of the GPD graph we first discussed.

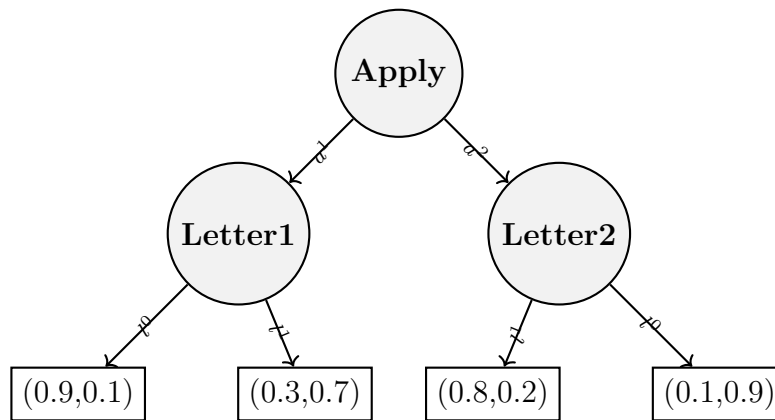
in this example there context-specific in-dependencies:

1. $J \perp_c L | a^1, s^1$
2. $J \perp_c L, S | a^0$
3. $J \perp_c L | s^1, A$

another example, assume in the case that the student have two letters:



this graph would map to the following context-specific CPD



in this case $(L1 \perp_c L2 | J, A)$

8.8.2 Multiplexer CPD

Chapter 9

Natural language processing

9.1 pre-processing

the problem here is how to extract features $\mathbf{X}$ from the a sentence.

for example how to classify a sentence being positive, or negative, assigning 0 for negative, and 1 for positive, starting for a preprocessed sentence how to turn it into a feature set X .

but there are unnecessary punctuation, conjugation, and stops that need to be get rid of, so first we need to pre-process our data-set as follows:

1. eliminate handles and URLs
2. tokenize the string into words
3. remove stop words such as “and, is, a, on, etc”
4. covert every word into it’s stem form
5. lower case transformation

9.2 Example: positive, negative classifier

given sentence s =“i love NLP, therefore i study it” how to classify s =[‘i’, ‘love’, ‘NLP’, ‘,’’, ‘therefore’, ‘i’, ‘study’, ‘it’] as positive or negative?.

for a set of strings $S = \{s_1, s_2, \dots, s_n\}$, matching each string against a vocabulary of all words to end up with two vectors of word frequency,

we create positive frequency vector $\text{freqs}(w,1)$, and negative frequency vector $\text{freqs}(w,0)$ such that w stand for sentence word, and such that $X_m = [1, \sum_w \text{freqs}(w,1), \sum_w \text{freqs}(w,0)]$

given a set of sentences, each is labeled for training as either positive, or negative. we mark each word in a positive-labeled sentence as positive, even if the word is negative, and conversely the opposite with negative-labeled sentences, we mark every word as negative.

first of all we create a set vocabulary V that includes all sets, or sentences S , $V = \{\text{words}(s_i) | s_i \in S\}$.

for example in training sets s_1, s_2 , $s_1 = \text{"i love NLP, therefore i study it"}$ labeled as positive, and $s_2 = \text{"society no longer in need for black magic, or superstition"}$ labeled negative, we can extract pos-neg features against vocabulary $V = [\text{'i'}, \text{'love'}, \text{'NLP'}, \text{' '}, \text{'therefore'}, \text{'study'}, \text{'it'}, \text{'society'}, \text{'no'}, \text{'longer'}, \text{'in'}, \text{'need'}, \text{'for'}, \text{'black'}, \text{'magic'}, \text{'or'}, \text{'superstition'}]$, pos-freqs = $[2, 1, 1, 1, 1, 1, 1, 0, 0, 0, \dots, 0]$, neg-freqs = $[0, 0, 0, 1, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1]$. $X_m = [1, 8, 11]$. for m training sets,

$$X_m = \begin{bmatrix} 1 & x_1^{(1)} & x_2^{(1)} \\ 1 & x_1^{(2)} & x_2^{(2)} \\ 1 & x_1^{(3)} & x_2^{(3)} \\ \dots & \dots & \dots \\ 1 & x_1^{(m)} & x_2^{(m)} \end{bmatrix} \text{pp}$$

9.3 Logistic regression classifier

let's see how this fits inside the Gradient Descent algorithm, for $\sigma(X^i, \theta^i) = \frac{1}{1+e^{-z}}$, we add the bias b to the X^i itself, and therefore $z = \theta^T X^i$.

therefore for a positive z we get $\sigma > 0.5$, and inversely for negative z we have $\sigma < 0.5$.

now we ready for Gradient Descent, given initial parameters θ , we predict, or evaluate the logistic:

$\sigma(X^i, \theta_j)$, then loss function $L(\hat{y}, y) = -[\hat{y} \log(\hat{y}) + (1-y) \log(1-\hat{y})]$, gradient $\nabla = \partial J / \partial \theta_j = \frac{X^T}{m}(\hat{y} - y)$, updating $\theta_j := \theta_j - \alpha \nabla$. iterate through gradient descent k times.

9.4 Naive Bayes classifier

The prior probability represents the underlying probability in the target population that a tweet is positive versus negative. In other words, if we had no specific information and blindly picked a tweet out of the population set, what is the probability that it will be positive versus that it will be negative? That is the "prior".

The prior is the ratio of the probabilities $\frac{P(D_{pos})}{P(D_{neg})}$.

We can take the log of the prior to rescale it, and we'll call this the logprior

$$\text{logprior} = \log \left(\frac{P(D_{pos})}{P(D_{neg})} \right) = \log \left(\frac{D_{pos}}{D_{neg}} \right)$$

Note that $\log(\frac{A}{B})$ is the same as $\log(A) - \log(B)$. So the logprior can also be calculated as the difference between two logs:

$$\text{logprior} = \log(P(D_{pos})) - \log(P(D_{neg})) = \log(D_{pos}) - \log(D_{neg})$$

To compute the positive probability and the negative probability for a specific word in the vocabulary, we'll use the following inputs:

$freq_{pos}$ and $freq_{neg}$ are the frequencies of that specific word in the positive or negative class. In other words, the positive frequency of a word is the number of times the word is counted with the label of 1.

N_{pos} and N_{neg} are the total number of positive and negative words for all documents (for all tweets), respectively. - V is the number of unique words in the entire set of documents, for all classes, whether positive or negative.

We'll use these to compute the positive and negative probability for a specific word using this formula:

$$P(W_{pos}) = \frac{freq_{pos} + 1}{N_{pos} + V}$$

$$P(W_{neg}) = \frac{freq_{neg} + 1}{N_{neg} + V}$$

Notice that we add the "+1" in the numerator for additive smoothing.

To compute the loglikelihood of that very same word, we can implement the following equations:

$$\text{loglikelihood} = \log \left(\frac{P(W_{pos})}{P(W_{neg})} \right)$$

$$p = \text{logprior} + \sum_i^N (\text{loglikelihood}_i)$$

Some words have more positive counts than others, and can be considered "more positive". Likewise, some words can be considered more negative than others.

One way for us to define the level of positiveness or negativeness, without calculating the log likelihood, is to compare the positive to negative frequency of the word.

Note that we can also use the log likelihood calculations to compare relative positivity or negativity of words.

We can calculate the ratio of positive to negative frequencies of a word.

Once we're able to calculate these ratios, we can also filter a subset of words that have a minimum ratio of positivity / negativity or higher.

Similarly, we can also filter a subset of words that have a maximum ratio of positivity / negativity or lower (words that are at least as negative, or even more negative than a given threshold).

$$\text{ratio} = \frac{pos_{words} + 1}{neg_{words} + 1}$$

In previous section a Logistic regression classified, but we can quick solve the same problem in much simpler algorithm through the evaluation of the likelihood of a sentence being positive matched against given Vocabulary table V .

Recall that conditional probability $p(w_i|pos) = \frac{p(w_i \cap pos)}{p(pos)}$, $p(pos) = freq(pos)/total$, $p(w_i \cap pos) = freq(w_i)/total$, then $p(w_i|pos) = freq(w_i)/freq(pos)$

Likelihood of a positive is defined as $\prod_{i=1}^m \frac{p(w_i|pos)}{p(w_i|neg)}$, if likelihood > 1 then sentence is positive, otherwise, it's negative.

to reduce the sensitivity of each word, and avoid getting 0 $p(w_i|class)$, $class \in \{pos, neg\}$ we modify the conditional probabilistic frequency using the so-called 'laplacian smoothing': $p(w_i|class) = \frac{freq(w_i|class)+1}{freq(class)+unique(V)}$, $freq(class)$ is defined as N_{class} , and $unique(V)$ is defined as N_V , for example $p(w_i|pos) = \frac{freq(w_i|pos)+1}{N_{pos}+N_V}$.

to keep the scale small as possible likelihood is replaced with log of likelihood coined with symbol $\lambda(w_i) = \log(\frac{p(w_i|pos)}{p(w_i|neg)})$, and a prior $= \log(\frac{p(pos)}{p(neg)})$, the classifier of a sentence W is equivalent to prior $+ \sum_{i=1}^m \lambda w_i$, and if $\lambda > 0$ then it's positive, and negative otherwise.

9.5 cosine similaritis

The cosine similarity function is:

$$\cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \|\mathbf{B}\|} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \sqrt{\sum_{i=1}^n B_i^2}}$$

A and B represent the word vectors and A_i or B_i represent index i of that vector.

Note that if A and B are identical, you will get $\cos(\theta) = 1$.

Otherwise, if they are the total opposite, meaning, $A = -B$, then you would get $\cos(\theta) = -1$.

If you get $\cos(\theta) = 0$, that means that they are orthogonal (or perpendicular).

Numbers between 0 and 1 indicate a similarity score.

Numbers between -1-0 indicate a dissimilarity score.

9.5.1 Euclidean distance

You will now implement a function that computes the similarity between two vectors using the Euclidean distance. Euclidean distance is defined as:

$$\begin{aligned} d(\mathbf{A}, \mathbf{B}) = d(\mathbf{B}, \mathbf{A}) &= \sqrt{(A_1 - B_1)^2 + (A_2 - B_2)^2 + \cdots + (A_n - B_n)^2} \\ &= \sqrt{\sum_{i=1}^n (A_i - B_i)^2} \end{aligned}$$

n is the number of elements in the vector

A and B are the corresponding word vectors.

The more similar the words, the more likely the Euclidean distance will be close to 0.

9.6 Principle Component Analysis (PCA)

PCA is a method that projects our vectors in a space of reduced dimension, while keeping the maximum information about the original vectors in their reduced counterparts. In this case, by *maximum information* we mean that the Euclidean distance between the original vectors and their projected siblings is minimal. Hence vectors that were originally close in the embeddings dictionary, will produce lower dimensional vectors that are still close to each other.

such that similar words will be clustered next to each other. For example, the words 'sad', 'happy', 'joyful' all describe emotion and are supposed to be near each other when plotted. The words: 'oil', 'gas', and 'petroleum' all describe natural resources. Words like 'city', 'village', 'town' could be seen as synonyms and describe a

Before plotting the words, you need to first be able to reduce each word vector with PCA into 2 dimensions and then plot it. The steps to compute PCA are as follows:

- Mean normalize the data
- Compute the covariance matrix of the data (Σ).
- Compute the eigenvectors and the eigenvalues of your covariance matrix
- Multiply the first K eigenvectors by the normalized data.

9.7 Machine Translation

Given dictionaries of English and French word embeddings you will create a transformation matrix ‘ $\mathbf{R}$ ’

Given an English word embedding, $\mathbf{e}$, you can multiply $\mathbf{eR}$ to get a new word embedding $\mathbf{f}$.

Both $\mathbf{e}$ and $\mathbf{f}$ are row vectors.

we can then compute the nearest neighbors to ‘ $\mathbf{f}$ ’ in the french embeddings and recommend the word that is most similar to the transformed word embedding.

Find a matrix ‘ $\mathbf{R}$ ’ that minimizes the following equation.

$$\arg \min_{\mathbf{R}} \|\mathbf{XR} - \mathbf{Y}\|_F$$

The Frobenius norm of a matrix A (assuming it is of dimension m, n) is defined as the square root of the sum of the absolute squares of its elements:

$$\|\mathbf{A}\|_F \equiv \sqrt{\sum_{i=1}^m \sum_{j=1}^n |a_{ij}|^2}$$

In the real world applications, the Frobenius norm loss:

$$\|\mathbf{XR} - \mathbf{Y}\|_F$$

is often replaced by it's squared value divided by m :

$$\frac{1}{m} \|\mathbf{XR} - \mathbf{Y}\|_F^2$$

where m is the number of examples (rows in $\mathbf{X}$).

The same R is found when using this loss function versus the original Frobenius norm.

The reason for taking the square is that it's easier to compute the gradient of the squared Frobenius.

The reason for dividing by m is that we're more interested in the average loss per embedding than the loss for the entire training set.

The loss for all training set increases with more words (training examples), so taking the average helps us to track the average loss regardless of the size of the training set.

9.7.1 Loss function L

The loss function will be squared Frobenius norm of the difference between matrix and its approximation, divided by the number of training examples m .

Its formula is:

$$L(X, Y, R) = \frac{1}{m} \sum_{i=1}^m \sum_{j=1}^n (a_{ij})^2$$

where a_{ij} is value in i th row and j th column of the matrix $\mathbf{XR} - \mathbf{Y}$.

The norm is always nonnegative (we're summing up absolute values), and so is the square.

When we take the square of all non-negative (positive or zero) numbers, the order of the data is preserved.

For example, if $3 > 2$, $3^2 > 2^2$

Using the norm or squared norm in gradient descent results in the same location of the minimum.

Squaring cancels the square root in the Frobenius norm formula. Because of the chain rule, we would have to do more calculations if we had a square root in our expression for summation.

Dividing the function value by the positive number doesn't change the optimum of the function, for the same reason as described above.

We're interested in transforming English embedding into the French. Thus, it is more important to measure average loss per embedding than the loss for the entire dictionary (which increases as the number of words in the dictionary increases).

9.7.2 gradient descent

Calculate the gradient of the loss with respect to transform matrix 'R'.

The gradient is a matrix that encodes how much a small change in 'R' affect the change in the loss function.

The gradient gives us the direction in which we should decrease 'R'

to minimize the loss.

m is the number of training examples (number of rows in X).

The formula for the gradient of the loss function $L(X, Y, R)$ is:

$$\frac{d}{dR}L(X, Y, R) = \frac{d}{dR}\left(\frac{1}{m}\|XR - Y\|_F^2\right) = \frac{2}{m}X^T(XR - Y)$$

9.7.3 fixed number of iterations

You cannot rely on training loss getting low – what you really want is the validation loss to go down, or validation accuracy to go up. And indeed - in some cases people train until validation accuracy reaches a threshold, or – commonly known as "early stopping" – until the validation accuracy starts to go down, which is a sign of over-fitting.

Why not always do "early stopping"? Well, mostly because well-regularized models on larger data-sets never stop improving. Especially in NLP, you can often continue training for months and the model will continue getting slightly and slightly better. This is also the reason why it's hard to just stop at a threshold – unless there's an external customer setting the threshold, why stop, where do you put the threshold?

Stopping after a certain number of steps has the advantage that you know how long your training will take - so you can keep some sanity and not train for months. You can then try to get the best performance within this time budget. Another advantage is that you can fix your learning rate schedule – e.g., lower the learning rate at 10

Pseudocode:

1. Calculate gradient g of the loss with respect to the matrix R .
2. Update R with the formula:

$$R_{\text{new}} = R_{\text{old}} - \alpha g$$

Where α is the learning rate, which is a scalar.

9.7.4 k-Nearest neighbors algorithm

k-NN is a method which takes a vector as input and finds the other vectors in the dataset that are closest to it.

The 'k' is the number of "nearest neighbors" to find (e.g. k=2 finds the closest two neighbors).

9.7.5 Searching for the translation embedding

Since we're approximating the translation function from English to French embeddings by a linear transformation matrix R , most of the time we won't get the exact embedding of a French word when we transform embedding e

of some particular English word into the French embedding space.

This is where k-NN becomes really useful! By using 1-NN with eR as input, we can search for an embedding f (as a row) in the matrix Y which is the closest to the transformed vector eR .

Note: Distance and similarity are pretty much opposite things.

We can obtain distance metric from cosine similarity, but the cosine similarity can't be used directly as the distance metric.

When the cosine similarity increases (towards 1), the "distance" between the two vectors decreases (towards 0).

We can define the cosine distance between u and v as

$$d_{\cos}(u, v) = 1 - \cos(u, v)$$

9.7.6 LSH and document search

first implement a more efficient version of k-nearest neighbors using locality sensitive hashing. then apply this to document search.

Process the tweets and represent each tweet as a vector (represent a document with a vector embedding).

Use locality sensitive hashing and k nearest neighbors to find tweets that are similar to a given tweet.

we will now implement locality sensitive hashing (LSH) to identify the most similar tweet.

Instead of looking at all 10,000 vectors, you can just search a subset to find its nearest neighbors

we can divide the vector space into regions and search within one region for nearest neighbors of a given vector.

9.7.7 Bag-of-words (BOW) document models

Text documents are sequences of words.

The ordering of words makes a difference. For example, sentences "Apple pie is

better than pepperoni pizza." and "Pepperoni pizza is better than apple pie"

have opposite meanings due to the word ordering.

However, for some applications, ignoring the order of words can allow us to train an efficient and still effective model.

This approach is called Bag-of-words document model.

Document embedding is created by summing up the embeddings of all words in the document.

If we don't know the embedding of some word, we can ignore that word.

9.7.8 Choosing the number of planes

Each plane divides the space to 2 parts.

So n planes divide the space into 2^n hash buckets.

We want to organize 10,000 document vectors into buckets so that every bucket has about 16 vectors.

For that we need $\frac{10000}{16} = 625$ buckets.

We're interested in n , number of planes, so that $2^n = 625$. Now, we can calculate $n = \log_2 625 = 9.29 \approx 10$.

In 3-dimensional vector space, the hyperplane is a regular plane. In 2 dimensional vector space, the hyperplane is a line.

Generally, the hyperplane is subspace which has dimension 1 lower than the original vector space has.

A hyperplane is uniquely defined by its normal vector.

Normal vector n of the plane π is the vector to which all vectors in the plane π are orthogonal (perpendicular in 3 dimensional case).

9.7.9 Getting the hash number for a vector

For each vector, we need to get a unique number associated to that vector in order to assign it to a "hash bucket".

Using Hyperplanes to split the vector space:

We can use a hyperplane to split the vector space into 2 parts.

All vectors whose dot product with a plane's normal vector is positive are on one side of the plane.

All vectors whose dot product with the plane's normal vector is negative are on the other side of the plane.

Encoding hash buckets:

For a vector, we can take its dot product with all the planes, then encode this information to assign the vector to a single hash bucket.

When the vector is pointing to the opposite side of the hyperplane than normal, encode it by 0.

Otherwise, if the vector is on the same side as the normal vector, encode it by 1.

If you calculate the dot product with each plane in the same order for every vector, you've encoded each vector's unique hash ID as a binary number, like $[0, 1, 1, \dots 0]$.

hash algorithm:

We've initialized hash table 'hashes' for you. It is list of $N_{universe}$ matrices, each describes its own hash table. Each matrix has N_{dims} rows and N_{planes} columns. Every column of that matrix is a N_{dims} dimensional normal vector for each of N_{planes} hyperplanes which are used for creating buckets of the particular hash table.

First multiply your vector 'v', with a corresponding plane. This will give you a vector of dimension N_{planes} .

You will then convert every element in that vector to 0 or 1.

You create a hash vector by doing the following: if the element is negative, it becomes a 0, otherwise you change it to a 1.

You then compute the unique number for the vector by iterating over N_{planes}

Then you multiply 2^i times the corresponding bit (0 or 1).

You will then store that sum in the variable $hash_{value}$.

$$hash = \sum_{i=0}^{N-1} (2^i \times h_i)$$

9.8 Probabilistic model of pronunciation and spelling

9.8.1 auto-correction

the misspelling is quite common in writing, and to transduct a word from the misspelled form to dictionary closest word, most relevant to the context for spelling, and pronunciation we utilize **Bayes Rule**, and **the noisy channel model**, and this problem can be divided into two categories:

1. word error detection: in which the algorithm is run on the word in isolation.
2. context error detection: where correction take place in a specific context.

80% of the misspelled words are caused by single-error misspellings: can be divided into four categories:

- insertion: mistyping the as ther
- deletion: mistyping the as th
- substitution: mistyping the as thw
- transposition mistyping the as hte

9.8.2 Bayesian inference model

given a noisy word through noisy channel, $\mathbf{O}$ as our observation, we need to match it to the nearest word in the dictionary.

we build a vocabulary $\mathbf{V}$, and our model ought to map noisy $\mathbf{O}$ to $\hat{w}$.

$$\hat{w} = \operatorname{argmax}_{w \in V} P(w|O)$$

$$\hat{w} = \operatorname{argmax}_{w \in V} \frac{P(O|w)P(w)}{P(O)}$$

since we iterate through the whole word set in the vocabulary $\mathbf{V}$ $P(O)$ is fixed, and we can ignore it, then:

$$\hat{w} = \operatorname{argmax}_{w \in V} P(O|w)P(w)$$

where $P(w)$ is the **prior**, $P(O|w)$ is the **likelihood** function.

$p(w)$ is the word frequency: $\frac{\text{frequency}(w)}{\text{size of the corpus}}$, to avoid getting zero frequency we use Laplacian smoothing:

$$p(w) = \frac{freq(w) + 1}{N + V}$$

such that V is the size of vocabulary in this context, and N is the size of the corpus.

there different algorithms for error correction, and processing, among those are **minimum edit distance**, **Viterbi forward**, **CYK**, **Earley**.

9.8.3 Minimum edit distance

It's a metric value between different noise channels for the same word, or weight for insertion, deletion, and substitution, weighting each by 1, but substitution by 2 (insertion+substitution), known as **Levenshtein** distance.

given two words target, and source, word distance can be calculated through Dynamic programming, laying out the target of length $\rightarrow n$ in the first column, and source of length $\rightarrow m$ in the first row, and creating matrix **distance** of size $\rightarrow n \times (n+1, m+1)$.

looping through each column i from $0 \rightarrow n$, and each row j from $0 \rightarrow m$:

$$distance[i, j] \leftarrow Min \begin{cases} (distance[i-1, j] + insertion-cost(target_j)) \\ distance[i-1, j-1] + subtraction-cost(source_j, target_i) \\ distance[i, j-1] + insertion-cost(source_j) \end{cases}$$

9.9 Grammar Weighted Automata

it's a weighted directed graph of finite automaton for language, to predict the probability of following word.

9.9.1 Markov chain

9.9.2 Hidden Markov Models HMMs

it's a special type of weighted Automata, in which previous states determine the current state.

the weights over the directed arrows can be loaded from a given corpus as the probability of word w_i followed by w_j , or in case of pronunciation, the probability of phone(p) p_i followed by p_j .

in tagged words we can classify each word in the corpus into specific group, and build the weighted graph for the classes instead.

a weighted automaton is consisting of a set of states $q = \langle q_0 q_1 q_2 \dots q_n \rangle$, and transition states $a_{01} a_{12} a_{23} \dots a_{n-1} a_n$, while the input to the machine is called the Observation and denoted by $O = (o_1 o_2 o_3 \dots o_t)$.

decoding problem: is the resolution of the underlying sequence that produce a certain observation.

word-detection is done through Bayesian inference $P(w|O) = P(O|w)p(w)$ if we ignore the denominator as discussed earlier.

this method has two important elements first the forward algorithm which is analogous to the Minimum Edit distance algorithm, yet more generalized, the latter can be seen as a special case, in the forward algorithm, the row do not just represent a sequence of characters, but does indicate the possibilities to reach to each state q_i from any previous state, and instead of the calculation of the minimum, here the sum of all probabilities in current state j $forward[t,j]$ after observing the first t observations, given the automaton λ of paths that lead to current state of aggregated, or in other words, the likelihood of the observations times the word probability $p(w)$ and this is calculated through multiplying three different factors:

1. previous path probability $forward[t-1,i]$.
2. transition probability a_{ij} .
3. observation likelihood b_{jt} that the current state j matches the observation symbol t . can take the range $[0,1]$, 1 if there is match, and 0 otherwise.

$$forward[t, j] = P(o_1, o_2, \dots, o_t, q_t = j | \lambda) P(w)$$

where $q_t = j$ means the probability that the t'th state in the sequence of states is state j (current state).

after initializing $forward[0,0]=1$.

$$forward[j, t] = forward[i, t - 1] * a[j, i] * b[j, o_t]$$

(note that i is index of previous state),

the problem with Forward algorithm is that it's run through a single channel. There is more efficient graph than that of forward algorithm which enable us to track multiple of words, or sentences moving from state to another, or a more general algorithm called Viterbi.

9.9.3 Viterbi Algorithm

it's consists of three main steps, **initialization** of viterbi probability, and path matrix, and viterbi **forward**, and viterbi **backward**.

it's a variation, or general implementation of the forward algorithm with multiple words/sentences running simultaneously.

we set up a probability matrix, where each column is set for time index t, and one row for each state in the Automata graph.

each column has a cell for each state q_i .

the evaluation of the $viterbi[t,j]$ is the same as in the forward algorithm.

there is a slight difference with the Forward algorithm, which is the viterbi maximizes the sum of all path to current state.

9.9.4 Part of Speech tagging (POS)

starting with the phenomenon: a single word can have different meanings in different sentences.

How to model such linguistic complexity in language processing?

look at the following examples:

- The whole team played well. [adverb]
- You are doing well for yourself. [adjective]
- Well, this assignment took me forever to complete. [interjection]
- The well is dry. [noun]
- Tears were beginning to well in her eyes. [verb]

the same word is used in different contexts to mean different things with different part of speech tags.

Algorithm:

1. calculate transition counts vector C_t from tag t_{i-1} to t_i to calculate $P(t_i|t_{i-1})$.
2. calculate emission counts vector C_e of tag t_i and word w_i to calculate $P(w_i|t_i)$.
3. tag counts is the count C of occurrence of specific tag.
4. create a Markov model of tag states, of transition probability from step (1), and create **transition matrix A** of all the states as follows: $\frac{C_t(t_{i-1}, t_i) + \alpha}{C(t_{i-1}) + \alpha N}$ smoothed to avoid zero, or undefined value, or in other words to calculate the probability of the relation that doesn't exist, constant α usually small value of 0.001, and N the number of tags.
5. create **emission matrix B** from step(2) for the probability of observation O at specific state t : $\frac{C_e(t_i, w_i) + \alpha}{C(t_i) + \alpha V}$. where V is the number of words in the dictionary.
6. the formula for the Viterbi matrix is different in the first column(first word) than the rest of the columns, in the previous column the previous value term is ignored!

7. Viterbi initialization: the previous value is set to the beginning tag (i.e str -s-) let's call it start-state, of index 0, or $start_{idx}$, also to avoid getting zero value we take the logarithm of the natural e, of the transition, and emission states: $\log(A[0, j] * \log[j, w_i]) = \log(A[0, j]) + \log[j, w_i]$ (where j in the current tag index in the list of tags of size N, and i is the previous state index, and in the case of initialization it's 0, $j \in N$), therefore the final initialization algorithm is:

$$V[j, 0] = \log(A[0, j]) + \log[j, w_i]$$

w_i is the current state word.

8. Viterbi forward:

$$V[j, i] = \operatorname{argmax}_{k \in N} (V[k, i - 1] + \log(A[k, j]) + \log[j, w_i])$$

9. Viterbi backward: starting for the last we pick the highest probability in the Viterbi matrix and record the indices of the corresponding tag, going backward up to the first column, add those tags in reverse order.

9.10 N-grams

9.10.1 Smoothing

Add-k smoothing was described as a method for smoothing of the probabilities for previously unseen n-grams.

9.10.2 Back-off

Back-off is a model generalization method that leverages information from lower order n-grams in case information about the high order n-grams is missing. For example, if the probability of an trigram is missing, use bigram information and so on.

9.10.3 Interpolation

The other method for using probabilities of lower order n-grams is the interpolation. In this case, you use weighted probabilities of n-grams of all orders every time, not just when high order information is missing.

Bibliography

Appendices

Appendix A

Introduction to probabilities

A.1 probabilities chain rule

the conditional probability of an event A given B is defined as $P(A|B) = \frac{P(A,B)}{P(B)}$, and therefore the joint probability is $P(A, B) = P(A|B)P(B)$.

applying this rule repeatedly to a set of random variables $X_1, X_2, \dots, X_n$ gives the **chain rule** (general product rule):

$$P(X_1, X_2, \dots, X_n) = P(X_1)P(X_2|X_1)P(X_3|X_1, X_2)\dots P(X_n|X_1, \dots, X_{n-1}) = \prod_{i=1}^n P(X_i|X_1, \dots, X_{i-1}).$$

the chain rule is the basis of sequence and language models, where the probability of a sentence $w_1\dots w_n$ factorizes as $\prod_{i=1}^n P(w_i|w_1, \dots, w_{i-1})$.

A.2 Naive Bayes

Bayes rule inverts a conditional probability: $P(y|x) = \frac{P(x|y)P(y)}{P(x)}$, where $P(y)$ is the prior, $P(x|y)$ the likelihood, and $P(y|x)$ the posterior.

the **naive** assumption is that the features $x = (x_1, \dots, x_n)$ are conditionally independent given the class y , so by the chain rule $P(x_1, \dots, x_n|y) = \prod_{i=1}^n P(x_i|y)$.

the classifier picks the most probable class (MAP estimate), and since $P(x)$ is constant across classes:

$$\hat{y} = \arg \max_y P(y) \prod_{i=1}^n P(x_i|y).$$

to avoid underflow from multiplying many small probabilities we work in log-space: $\hat{y} = \arg \max_y [\log P(y) + \sum_{i=1}^n \log P(x_i|y)]$.

Appendix B

Covariance

the covariance of two random variables measures how they vary together:

$$\text{Cov}(X, Y) = E[(X - E[X])(Y - E[Y])] = E[XY] - E[X]E[Y].$$

for a random vector $X \in R^n$ with mean $\mu = E[X]$, the **covariance matrix** is $\Sigma = E[(X - \mu)(X - \mu)^T]$, with entries $\Sigma_{ij} = \text{Cov}(X_i, X_j)$; it is symmetric and positive semi-definite, and its diagonal holds the variances $\text{Var}(X_i)$.

the (Pearson) **correlation** normalizes covariance by the standard deviations, $\rho_{XY} = \frac{\text{Cov}(X, Y)}{\sigma_X \sigma_Y} \in [-1, 1]$, removing the dependence on scale.

given a data matrix of m samples, the **sample covariance** is $\hat{\Sigma} = \frac{1}{m-1} \sum_{i=1}^m (x^{(i)} - \bar{x})(x^{(i)} - \bar{x})^T$, where $\bar{x}$ is the sample mean (the $m-1$ denominator gives an unbiased estimate).

Appendix C

Single Value Decomposition

any real matrix $A \in R^{m \times n}$ can be factorized as

$$A = U\Sigma V^T,$$

where $U \in R^{m \times m}$ and $V \in R^{n \times n}$ are orthogonal ($U^T U = I$, $V^T V = I$), and $\Sigma \in R^{m \times n}$ is diagonal with non-negative entries $\sigma_1 \geq \sigma_2 \geq \dots \geq 0$ called the **singular values**.

the columns of U (left singular vectors) are the eigenvectors of AA^T , the columns of V (right singular vectors) are the eigenvectors of $A^T A$, and the singular values are the square roots of the shared eigenvalues, $\sigma_i = \sqrt{\lambda_i}$.

keeping the largest k singular values gives the best rank- k approximation $A_k = \sum_{i=1}^k \sigma_i u_i v_i^T$ in the least-squares sense (Eckart–Young theorem), which is the basis of dimensionality reduction and PCA (where SVD is applied to the mean-centered data / covariance matrix).

SVD also yields the Moore–Penrose **pseudo-inverse** $A^+ = V\Sigma^+ U^T$, used to solve least-squares problems $\min_x \|Ax - b\|$, where Σ^+ inverts the non-zero singular values.

Appendix D

Exponentially weighted averages

if we have a stochastic function over vector Θ , to smooths it's overshoot a bit, we can take the averages over a window of size k , but for large sized vector Θ , it's expensive to keep all values at memory, and repeatedly run the average function over a small window, instead we can keep just the previous value through weighted averages.

the weighted averages for $\Theta = (\theta^1, \theta^2, \dots, \theta^n)$ the weighted averaged θ^i :
 $V\theta^i = \beta V\theta^{i-1} + (1 - \beta)\theta^i$.

there is a bias in the weighted averages in the initial steps, we can compensate for that through **bias correction** $V\theta^i = \frac{V\theta^i}{(1-\beta^t)}$. where t is a positive integer of the first initial values, for example if we doing bias correction for the first 10 values, then $t = 10$.

D.0.1 How to choose the value β ?

it's more efficient to use average function over a window of inputs, but it's expensive in a large input set, in fact the β value is a control of window size as it tend to fade away as the exponential gets bigger.

if we expand the $dV\theta^n$:

$$(1 - \beta)\theta^n + \beta(1 - \beta)^2\theta^{n-1} + \dots + \beta(1 - \beta)^n\theta^1$$

and we know that $(1 - \beta)^{\frac{1}{\beta}} = \frac{1}{e} = 0.367$ then $\frac{1}{1-\beta}$ iterations this weighting term will be $\frac{1}{e}$, meaning closer to zero, and then we can interpret the weight decay algorithm as the function that take the average over the last $\frac{1}{1-\beta}$ elements.

Appendix E

smoothing (add-k, and add-one Laplacian)

the maximum-likelihood estimate of an n-gram probability, $P(w_i|w_{i-1}) = \frac{\text{count}(w_{i-1}, w_i)}{\text{count}(w_{i-1})}$, assigns zero probability to any n-gram that was never seen in training, which makes the probability of a whole sentence collapse to zero. smoothing moves a little probability mass from seen events to unseen ones.

add-one (Laplace) smoothing adds a count of one to every possible n-gram:

$$P_{Laplace}(w_i|w_{i-1}) = \frac{\text{count}(w_{i-1}, w_i) + 1}{\text{count}(w_{i-1}) + V},$$

where V is the vocabulary size (the number of possible words), added to the denominator so the distribution still sums to one.

add-k smoothing generalizes this to a fractional pseudo-count k (typically $0 < k < 1$, tuned on a dev set):

$$P_{add-k}(w_i|w_{i-1}) = \frac{\text{count}(w_{i-1}, w_i) + k}{\text{count}(w_{i-1}) + kV}.$$

add-one is often too aggressive for large vocabularies (it steals too much mass from observed n-grams); back-off and interpolation are usually preferred in practice, but add-k remains a simple, useful baseline.

Appendix F

Kernels, and Convolution functions

the **convolution** of two functions is $(f * g)(t) = \int f(\tau)g(t - \tau) d\tau$, and in the discrete case $(f * g)[n] = \sum_m f[m]g[n - m]$; it slides one function over another, at each shift multiplying and summing the overlap.

in convolutional networks the operation applied is really 2D cross-correlation of an image I with a kernel (filter) K :

$$(I * K)[i, j] = \sum_u \sum_v I[i + u, j + v] K[u, v],$$

where the learned kernel weights K detect local patterns (edges, textures) with parameter sharing across the whole image.

in the kernel-methods sense (e.g. SVMs) a **kernel function** $k(x, x') = \langle \phi(x), \phi(x') \rangle$ computes an inner product in some high-dimensional feature space ϕ without ever forming $\phi(x)$ explicitly — the *kernel trick*.

common kernels are the linear $k(x, x') = x^T x'$, the polynomial $k(x, x') = (x^T x' + c)^d$, and the Gaussian / RBF $k(x, x') = \exp\left(-\frac{\|x - x'\|^2}{2\sigma^2}\right)$. a valid kernel must satisfy Mercer's condition: its Gram matrix $K_{ij} = k(x^{(i)}, x^{(j)})$ is symmetric positive semi-definite.